CEH v13 AI (312-50) Study Guide

Module 03: Scanning Networks

Exam-focused study guide built from the official theory video screenshots. ADDED CONTEXT marks content from outside the screenshots; verify marks facts to double check.

Table of Contents

1. Module at a Glance 4
2. Big Picture Map 6
3. Topic-by-Topic Guide 7
   - 3.1 Learning Objectives of Module 03 7
   - 3.2 Overview of Network Scanning 7
   - 3.3 TCP Communication Flags 9
   - 3.4 TCP/IP Communication (Handshake and Termination) 12
   - 3.5 Host Discovery Techniques 14
   - 3.6 Port Scanning Techniques 18
   - 3.7 Service Version Discovery 30
   - 3.8 How to Identify Target System OS 32
   - 3.9 Packet Fragmentation 34
   - 3.10 Source Port Manipulation 37
   - 3.11 IP Address Spoofing 39
   - 3.12 Creating Custom Packets 42
   - 3.13 OS Discovery / Banner Grabbing 44
   - 3.14 Scanning Beyond IDS and Firewall (Overview) 47
   - 3.15 Source Routing 50
   - 3.16 IP Address Decoy 52
   - 3.17 MAC Address Spoofing 54
   - 3.18 Randomizing Host Order and Sending Bad Checksums 57
   - 3.19 Port Scanning Countermeasures 60
   - 3.20 IP Spoofing Countermeasures 61
4. Tool Encyclopedia 64
   - 4.1 Core Scanners and Packet Crafters 64
   - 4.2 Host and Ping Sweep Tools 64
   - 4.3 OS Discovery, Banner Grabbing, and Packet Analysis 65
   - 4.4 Evasion and Anonymity Aids 65
5. Command Sheet 67
   - 5.1 Nmap Host Discovery Flags 67
   - 5.2 Nmap Port Scanning Flags 68
   - 5.3 Nmap Service and OS Detection 69
   - 5.4 Nmap Evasion Flags 70
   - 5.5 Hping3 71
   - 5.6 Nmap Output and Timing Options (ADDED CONTEXT) 72
   - 5.7 Reading OS Clues in Wireshark and tcpdump (ADDED CONTEXT) 73
   - 5.8 Netcat and Telnet Banner Grabbing (ADDED CONTEXT) 74
6. Ports, Protocols, Standards and Acronyms 75
   - 6.1 Ports Seen in This Module 75
   - 6.2 Protocol Layer Reference 75
   - 6.3 TCP Flags Quick Reference 76
   - 6.4 ICMP Types Used in Scanning 76
   - 6.5 Rare Protocols, Short Introductions 76

6.6 Acronyms 77
7. Attack vs Countermeasure and Look-alike Comparisons 78
7.1 Scanning Technique vs Countermeasure 78
7.2 Look-alike Pairs 79
8. Exam Cheat Sheet 80
8.1 Keyword -> Answer 80
8.2 Must-memorize Lists and Numbers 81
8.3 "Pick the BEST option" Rules for This Module 81
9. Practice Questions 83
10. Flashcards 96
11. Glossary 99
12. Coverage Report 101
12.1 Page to Section Map 101
12.2 [UNCLEAR] Items 101
12.3 Items to Double Check 102
12.4 Confirmation 102

Source: 10 pages of screenshots from the official Module 03 theory videos (19 slides, numbered 2 to 42). Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small to read with full confidence. The screenshots show only part of the official module, so the added context fills the gaps the exam commonly tests (Nmap flags, hping3, port states, banner grabbing tools).

1. Module at a Glance

Module: 03 - Scanning Networks (EC-Council CEH v13 AI).

What this module is (5 lines):

1. It defines network scanning: finding live hosts, open ports, running services, and the operating system of a target, right after footprinting.
2. It reviews the TCP flags and the three-way handshake, because every scan technique is built on how TCP answers a probe.
3. It covers host discovery (ping scans), port scanning (connect, SYN, Xmas, FIN, NULL, Maimon, ACK, window, idle, UDP, SCTP, SSDP, IPv6), service version discovery, and OS discovery (TTL, window size, banner grabbing).
4. It shows how attackers scan beyond an IDS or firewall: fragmentation, source routing, source port manipulation, decoys, IP and MAC spoofing, custom packets, random host order, bad checksums, proxies, and anonymizers.
5. It ends with countermeasures for port scanning and for IP spoofing.

Why it matters: Footprinting gives names. Scanning turns names into targets: which machines are alive, which doors are open, and which software sits behind each door. Nmap and Hping3 flags are among the most tested items in the whole CEH exam.

What the exam usually asks from it:

- Which Nmap flag runs which scan (-sS, -sT, -sX, -sF, -sN, -sA, -sW, -sI, -sU) and which host discovery flag does what (-PE, -PP, -PM, -PR, -PU, -PS, -PA, -PO, -sn, -Pn).
- What response a closed, open, or filtered port gives to each scan type.
- Which flags are set in an Xmas, FIN, NULL, or Maimon probe.
- How to guess an OS from TTL and TCP window size.
- Which technique evades an IDS or firewall (fragmentation, decoys, spoofing, source port, source routing).
- What an Hping3 command does (for example the -a option spoofs the source address).
- Which countermeasure stops which technique (ingress and egress filtering for spoofing, firewall rules for scans).

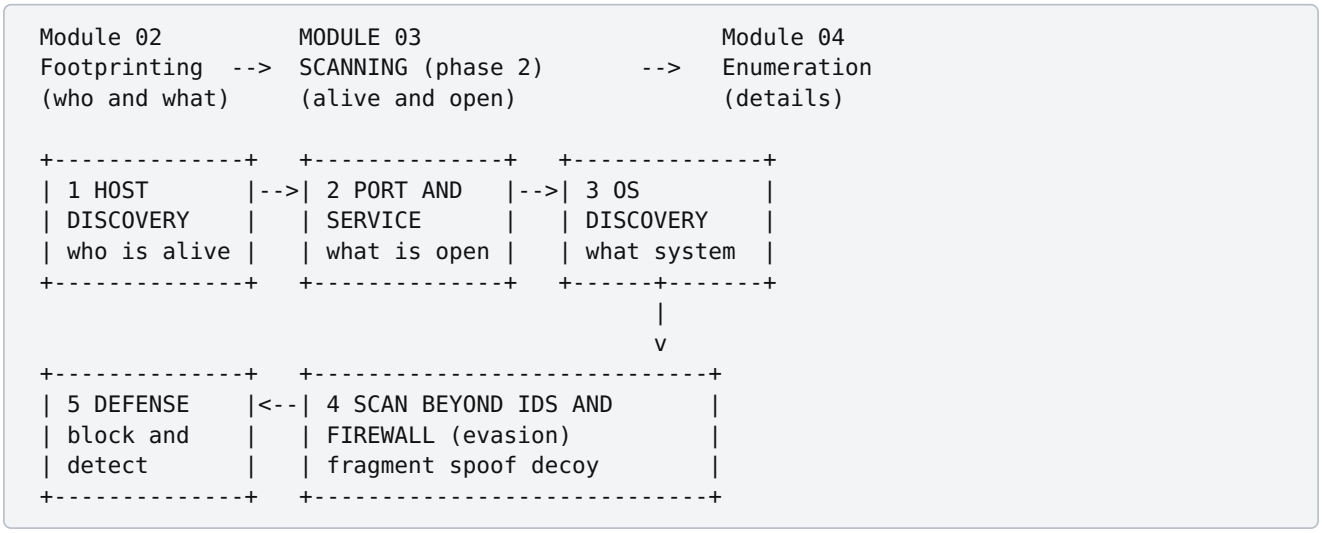
The 10 things to remember most:

1. Scanning is phase 2 of the five hacking phases (after reconnaissance). It finds live hosts, open ports and services, and the OS.
2. The six TCP flags: URG, ACK, PSH, RST, SYN, FIN. SYN starts a connection, FIN ends it politely, RST kills it at once.
3. Three-way handshake: SYN -> SYN/ACK -> ACK. The ACK number is the received sequence number plus 1.

4. Host discovery flags: -sn (ping scan only, no port scan), -PE echo, -PP timestamp, -PM address mask, -PR ARP, -PU UDP, -PS TCP SYN, -PA TCP ACK, -PO IP protocol, -Pn (skip discovery).
5. Port scan flags: -sT connect, -sS half-open (stealth), -sX Xmas, -sF FIN, -sN NULL, -sM Maimon, -sA ACK, -sW window, -sI idle, -sU UDP, -sY and -sZ SCTP, -6 IPv6.
6. Xmas = FIN + PSH + URG. NULL = no flags. FIN, NULL, and Xmas: an open port stays silent, a closed port answers RST. They fail against Windows, which answers RST for every port.
7. ACK scan maps firewall rules (filtered or unfiltered). It cannot tell open from closed.
8. Idle scan (-sI zombie) hides the scanner's own IP by using a zombie host's IP ID counter.
9. OS clues: TTL 64 = Linux and FreeBSD, 128 = Windows, 255 = Cisco, Solaris, OpenBSD, AIX. -sV finds service versions. -O (added) guesses the OS.
10. Evasion flags: -f fragments, -g sets source port, -D decoys, --spoof-mac, --randomize-hosts, --badsum. Hping3 -a spoofs the source IP. Defenses: ingress and egress filtering, firewall and IDS rules, ICMP filtering, patched firmware.

2. Big Picture Map

The module follows the order of a real scan: find who is alive, find what is open on them, find what they run, then try to do all of that without being caught. Defense closes the loop.



How each stage maps to the questions it answers:

Stage	Question answered	Main techniques	Main tools
1 Host discovery	Which IPs are alive?	ICMP, ARP, UDP, TCP, IP protocol pings	Nmap, Angry IP Scanner
2 Port scanning	Which ports are open?	Connect, SYN, Xmas, FIN, ACK, idle, UDP	Nmap, Hping3, Metasploit
2 Service discovery	Which software and version?	Version detection (-sV)	Nmap, Zenmap
3 OS discovery	Which operating system?	TTL, window size, banner grabbing	Wireshark, Nmap
4 Evasion	How to avoid IDS or firewall?	Fragments, decoys, spoofing, proxies	Nmap, Hping3, Packet Builder
5 Defense	How to stop scans?	Filtering, rules, updates	Firewall, IDS

The screenshots follow this order loosely. This guide keeps the order of your screenshots so you can match it with the videos.

3. Topic-by-Topic Guide

The topics below follow the order of the screenshots.

3.1 Learning Objectives of Module 03

What it is (simple words): The opening slide lists six goals for the module. They are also the skeleton of this guide.

1. Explain network scanning concepts.
2. Demonstrate various scanning techniques for host discovery.
3. Demonstrate various scanning techniques for port and service discovery.
4. Demonstrate various scanning techniques for OS discovery.
5. Demonstrate various techniques for scanning beyond IDS and firewall.
6. Explain network scanning countermeasures.

How the objectives map to the topics:

Objective	Topics in this guide
1 Concepts	3.2 overview, 3.3 TCP flags, 3.4 TCP/IP communication
2 Host discovery	3.5 host discovery techniques
3 Port and service discovery	3.6 port scanning, 3.7 service version discovery
4 OS discovery	3.8 identify OS, 3.13 banner grabbing
5 Beyond IDS and firewall	3.9 to 3.12, 3.14 to 3.18
6 Countermeasures	3.19 port scanning, 3.20 IP spoofing

EXAM FOCUS: The objectives show the question pools: concepts, host discovery, port and service discovery, OS discovery, evasion, and countermeasures. Expect questions from every pool.

Memory trick: "Can Hackers Please Obey Every Command?" -> Concepts, Hosts, Ports, OS, Evasion, Countermeasures.

3.2 Overview of Network Scanning

What it is (simple words): Network scanning is a set of procedures used for identifying hosts, ports, and services in a network. It is one of the components of information gathering. An attacker uses it to create a profile of the target organization.

- **Host:** any device with an IP address on a network (server, PC, router, printer, camera).
- **Live host:** a host that is switched on and answers probes.

- **Port:** a numbered door (0 to 65535) on a host. Each network service listens on one or more ports.
- **Service:** a program that listens on a port, such as a web server on port 80.
- **Probe:** a packet sent only to see how the target answers.

Analogy: Footprinting studied the building from the street. Scanning walks the corridors. It checks which rooms are occupied (live hosts), which doors are unlocked (open ports), and who is working inside (services and OS).

How it works (numbered steps):

1. The attacker sends TCP/IP probes to the network.
2. The network (its hosts and devices) sends back data.
3. The attacker reads the replies: who answered, from which port, with which flags, and with which banner.
4. The result is a list of live IP addresses, open ports, services, and likely OS.



Objectives of network scanning (from the slide):

1. To discover live hosts, IP addresses, and open ports of live hosts.
2. To discover operating systems and system architecture.
3. To discover services running on hosts.
4. To discover vulnerabilities in live hosts.

Tools named on the slide: Nmap, Hping3, Metasploit, NetScanTools Pro.

ADDED CONTEXT: three kinds of scanning. Port scanning finds open ports and services on a host. Network scanning finds live hosts and network layout. Vulnerability scanning checks hosts for known weaknesses (Module 05). The exam may ask you to pick which kind fits a sentence.

ADDED CONTEXT: the CEH scanning methodology (verify the exact names).

1. Check for live systems (host discovery).
2. Check for open ports (port scanning).
3. Scan beyond IDS and firewall.
4. Perform banner grabbing (OS and service discovery).
5. Scan for vulnerabilities.
6. Draw network diagrams of vulnerable hosts.
7. Prepare proxies.

Your screenshots cover steps 1 to 4 and 7 (as proxies and anonymizers). Steps 5 and 6 belong to later modules and the full courseware.

> **TRAP:** Scanning is not the same as footprinting. Footprinting uses public sources and can be passive. Scanning sends packets to the target, so it is always active.

> **EXAM TIP:** Authorization matters. A penetration tester scans only the IP ranges named in the signed scope. Scanning without written permission can be a crime in many countries.

Real-world use: A tester has a Whois result that says the client owns 203.0.113.0/24. She scans that range and finds 14 live hosts, one with an open Telnet port. Result: the client closes Telnet and moves to SSH.

Tools and commands used here: Nmap, Hping3, Metasploit, NetScanTools Pro (details in sections 4 and 5).

AI angle: An LLM (ChatGPT) or a command-line tool such as ShellGPT (sgpt) can write the scan command from plain English and explain the output. The screenshots do not show AI slides for this module, so this is ADDED CONTEXT. Example prompt: "Write an Nmap command that finds live hosts in 10.10.1.0/24 without scanning ports, then explain each flag." Always read the command before running it, and never paste in targets you are not allowed to scan.

Countermeasures and defense: Firewall and IDS rules that detect and block probes (see 3.19).

EXAM FOCUS:

- How it is asked: "Which of the following is an objective of network scanning?" or "Which tool is used for network scanning?"
- Keywords: "identify hosts, ports, and services" -> network scanning. "Profile of the target organization" -> information gathering.
- Look-alike: scanning vs enumeration. Scanning finds hosts, ports, and services. Enumeration (Module 04) extracts user names, shares, and other details from those services.

Memory trick: Scanning finds "H-P-S-O-V": Hosts, Ports, Services, OS, Vulnerabilities.

3.3 TCP Communication Flags

What it is (simple words): TCP (Transmission Control Protocol) is the reliable, connection-based transport protocol of the internet. Standard TCP communications are controlled by flags in the TCP packet header. A flag is a single bit (0 or 1) that tells the receiver what the packet means.

Analogy: Flags are hand signals in a phone call. SYN is "hello, can we talk?", ACK is "I heard you", PSH is "say it now, do not wait", URG is "this part is urgent", FIN is "goodbye, I am finished", RST is "I hang up, this call is wrong".

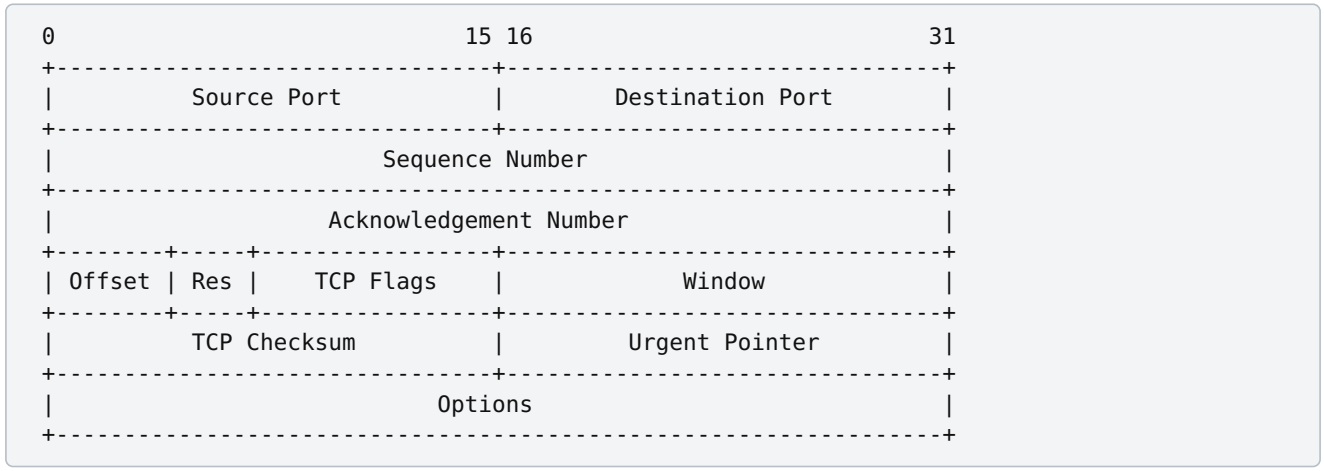
The six flags on the slide:

Flag	Full name	Meaning from the slide	Hex value	Easy picture
URG	Urgent	Data in the packet is processed immediately	0x20	Priority lane
ACK	Acknowledgement	Acknowledges receipt of a packet	0x10	"Got it"
PSH	Push	Sends all buffered data immediately	0x08	Flush now
RST	Reset	Resets a connection	0x04	Hang up
SYN	Synchronize	Initiates a connection between hosts	0x02	"Hello"
FIN	Finish	There will be no further transmissions	0x01	"Goodbye"

The slide groups them as URG, FIN, RST in the top row and PSH, ACK, SYN in the bottom row. Header order is URG, ACK, PSH, RST, SYN, FIN.

ADDED CONTEXT: The hex values are the bit positions inside the flags byte. Scanners combine them. Xmas = FIN + PSH + URG = 0x01 + 0x08 + 0x20 = 0x29. NULL = 0x00. Two more flags exist in modern TCP, CWR (0x80) and ECE (0x40), used for congestion control. The exam rarely asks about them.

The TCP header layout (from the slide):



Field	Plain meaning	Why scanners care
Source port, destination port	Sender's and receiver's door numbers	The port being probed
Sequence number	Position of this data in the stream	Spoofing needs a guess of it

Field	Plain meaning	Why scanners care
Acknowledgement number	Next byte the receiver expects	Proves a reply is real
Offset	Length of the TCP header	Fragment tricks split the header
Res	Reserved bits	Normally zero
TCP flags	The six flag bits	Defines the scan type
Window	How much data the receiver accepts	Helps guess the OS (see 3.8)
TCP checksum	Error check for the segment	Bad checksums reveal firewalls (see 3.18)
Urgent pointer	Where urgent data ends	Used only with URG
Options	Extra settings (for example MSS)	OS fingerprinting reads these

Each row of the header is 32 bits wide (bit 0 to bit 31).

Real-world use: A tester sends a packet with only the FIN flag to port 22. The target sends nothing back. Because the flag combination is unusual, a firewall that only blocks SYN packets may let it through. That is the idea behind FIN, NULL, and Xmas scans (3.6).

Tools and commands used here: Wireshark and tcpdump (to read flags), Nmap and Hping3 (to set flags).

AI angle: Paste a packet capture line into an LLM and ask: "Which TCP flags are set in this packet and what does it mean for the connection?" The model can decode flag letters quickly. Check its answer against the hex values above.

Countermeasures and defense: A stateful firewall tracks connections and drops packets whose flags do not fit the connection state (for example ACK packets with no matching SYN). An IDS alerts on illegal flag combinations such as SYN+FIN or all flags set.

EXAM FOCUS:

- How it is asked: "Which flag resets a connection?" -> RST. "Which flag initiates a connection?" -> SYN. "Which flag sends buffered data immediately?" -> PSH.
- Keywords: "no further transmissions" -> FIN. "Processed immediately" -> URG. "Acknowledges receipt" -> ACK.
- Trap: FIN ends a connection politely after the data is sent. RST ends it abruptly and discards data. PSH is about speed of delivery, not about ending anything.
- Look-alike: URG (priority data inside the stream) vs PSH (do not wait, deliver the buffer now).

Memory trick: Header order "Unskilled Attackers Pester Real Security Folks" -> URG, ACK, PSH, RST, SYN, FIN.

3.4 TCP/IP Communication (Handshake and Termination)

What it is (simple words): Before two hosts exchange data, TCP sets up a session with a three-way handshake. When they finish, TCP closes the session with a termination exchange. Port scanners study how a host answers these steps.

- **Sequence number (SEQ):** a counter that numbers the bytes a host sends.
- **Acknowledgement number (ACK#):** the next sequence number the receiver expects. It equals the received sequence number plus 1 for a SYN or FIN.
- **Session:** the period between connection setup and teardown.

Analogy: A phone call. "Hello, can you hear me?" (SYN). "Yes, I can hear you, can you hear me?" (SYN+ACK). "Yes, I can" (ACK). The call is open. At the end: "I am done talking" (FIN), "OK" (ACK), "I am done too" (FIN), "OK, bye" (ACK).

Session establishment (three-way handshake), example from the slide: Client Bill (10.0.0.2:21) talks to server Sheela (10.0.0.3:21).

```
Client Bill 10.0.0.2          Server Sheela 10.0.0.3
    |                                          |
    |--- 1 SYN  SEQ=10 --------------------------->|
    |                                          |
    |<-- 2 SYN+ACK  ACK=11  SEQ=142 ---------------|
    |                                          |
    |--- 3 ACK  ACK=143  SEQ=11 ------------------>|
```

What each packet says in plain words:

1. SYN: "Can we talk on port 21? Are you open?"
2. SYN+ACK: "OK, I am open on port 21, let us talk."
3. ACK: "OK, thanks." The session is now open.

How to read the numbers:

1. Client sends SYN with SEQ 10.
2. Server answers SYN+ACK. Its ACK is 11 (client SEQ 10 plus 1). Its own SEQ is 142.
3. Client sends ACK. Its ACK is 143 (server SEQ 142 plus 1). Its SEQ is 11 (the next byte after its SYN).

Session termination, example from the slide:

```
Client Bill                    Server Sheela
    |                                          |
    |--- 1 FIN  SEQ=50 --------------------------->|
    |                                          |
    |<-- 2 ACK  ACK=51  SEQ=170 ------------------|
    |                                          |
    |<-- 3 FIN  SEQ=171 --------------------------|
    |                                          |
    |--- 4 ACK  ACK=172  SEQ=51 ------------------>|
```

What each packet says in plain words:

1. FIN: "I am done with the data transfer."
2. ACK: "OK, I received your termination request."
3. FIN: "I have received all the data sent, I am done too."
4. ACK: "OK, thanks." The session is closed.

The termination uses four packets: FIN, ACK, FIN, ACK.

ADDED CONTEXT: reading the same handshake in tcpdump (illustrative).

```
10.0.0.2.51514 > 10.0.0.3.21: Flags [S], seq 10, win 64240
10.0.0.3.21 > 10.0.0.2.51514: Flags [S.], seq 142, ack 11, win 65535
10.0.0.2.51514 > 10.0.0.3.21: Flags [.], ack 143
```

tcpdump flag letters: S = SYN, . = ACK, F = FIN, R = RST, P = PSH, U = URG. So [S.] means SYN+ACK. Real captures show random starting sequence numbers (or relative numbers starting at 0). The client port on the slide is shown as 21 for simplicity; in real life the client uses a random high port (an ephemeral port such as 51514).

What this means for scanning:

Probe sent	Reply	Meaning
SYN	SYN+ACK	Port is open
SYN	RST (or RST+ACK)	Port is closed
SYN	Nothing or ICMP unreachable	Port is filtered
Connect scan after SYN+ACK	ACK, then RST	Full connection made, then closed
SYN scan after SYN+ACK	RST only	Half-open, connection never completes

Real-world use: A tester looks at a Wireshark capture. The server sends SYN+ACK but the client answers RST instead of ACK. That pattern is a SYN (half-open) scan, because the scanner never finishes the handshake.

Tools and commands used here: Wireshark, tcpdump, Nmap (-sT and -sS).

AI angle: Ask an LLM: "Explain this three-packet capture and tell me whether it is a full connect scan or a half-open scan." Use the table above to check the answer.

Countermeasures and defense: Use SYN cookies and connection limits to protect against half-open floods (ADDED CONTEXT). Log failed connections and alert on many SYNs from one source.

EXAM FOCUS:

- How it is asked: "In the three-way handshake, what does the server send after receiving SYN?" -> SYN+ACK. "What is the ACK number if the client SEQ is 10?" -> 11.
- Keywords: "half-open" -> the third step (ACK) is never sent. "Full open" or "connect" -> all three steps are completed.
- Trap: the termination is four steps, not three.
- Look-alike: FIN ends politely with ACKs on both sides. RST ends at once with no reply needed.

Memory trick: Start: "SYN, SYN-ACK, ACK" -> "Sally Sends Alice A kiss". End: "FIN, ACK, FIN, ACK".

3.5 Host Discovery Techniques

What it is (simple words): Host discovery techniques are used to identify the active (live) systems in the network. The scanner sends a probe to every IP address in a range. Any valid answer proves that a host is alive at that address. This step is often called a ping scan, but it goes beyond the classic ICMP ping.

- **ICMP (Internet Control Message Protocol):** a network-layer protocol that carries control and error messages. Ping uses it.
- **ARP (Address Resolution Protocol):** a local-network protocol that asks "who has this IP address?" and learns the MAC address of the answer.
- **MAC address:** the hardware address of a network card.
- **Ping sweep:** a ping sent to every address in a range, one after the other.

Analogy: Walking along a street and ringing every doorbell to learn which houses have someone at home. You do not try the doors yet. You only make a list of the occupied houses.

How it works (numbered steps):

1. Choose a target range, for example 10.10.1.0/24 (256 addresses).
2. Pick the probe type: ICMP, ARP, UDP, TCP SYN, TCP ACK, or IP protocol.
3. Send one probe to each address.
4. Mark every address that answers as live.
5. Pass only the live addresses to the next step, port scanning (3.6).

Host discovery techniques on the slide:

```
HOST DISCOVERY TECHNIQUES
|
+-- ICMP Ping Scan
|   +-- ICMP ECHO Ping            nmap -sn -PE TARGET
|   +-- ICMP Timestamp Ping       nmap -sn -PP TARGET
|   +-- ICMP Address Mask Ping    nmap -sn -PM TARGET
|   +-- ICMP ECHO Ping Sweep      nmap -sn -PE RANGE
+-- ARP Ping Scan                 nmap -sn -PR TARGET
+-- UDP Ping Scan                 nmap -sn -PU TARGET
+-- TCP Ping Scan
|   +-- TCP SYN Ping              nmap -sn -PS TARGET
|   +-- TCP ACK Ping              nmap -sn -PA TARGET
+-- IP Protocol Ping Scan         nmap -sn -PO TARGET
```

The techniques explained:

Technique	Flag	Probe sent	Host is alive if...	Best use
ICMP echo ping	-PE	ICMP type 8	ICMP type 0 reply	Simple internal checks
ICMP timestamp ping	-PP	ICMP type 13	ICMP type 14 reply	Echo blocked, others open
ICMP address mask ping	-PM	ICMP type 17	ICMP type 18 reply	Echo blocked, others open
ICMP echo ping sweep	-PE range	Type 8 to many IPs	Replies from many IPs	Map a whole subnet
ARP ping	-PR	ARP request	ARP reply	Local LAN, most reliable
UDP ping	-PU	UDP datagram	ICMP port unreachable or UDP reply	Where TCP and ICMP are blocked
TCP SYN ping	-PS	TCP SYN	SYN/ACK or RST	Firewall blocks ICMP
TCP ACK ping	-PA	TCP ACK	RST	Firewall blocks SYN
IP protocol ping	-PO	IP packets with set protocol numbers	Reply or ICMP protocol unreachable	Unusual filters

ICMP ping scans (echo, timestamp, address mask)

An ICMP echo request (type 8) asks "are you there?". A live host answers with an echo reply (type 0). Many firewalls block echo, so Nmap also offers two older ICMP queries: timestamp (request type 13, reply type 14) and address mask (request type 17, reply type 18). Administrators who block echo often forget these two. A ping sweep is the same echo ping sent to every address in a range.

ADDED CONTEXT: the ICMP types the exam likes.

Type	Name	Used for
0	Echo reply	Answer to ping
3	Destination unreachable	Closed UDP port (code 3), blocked traffic
8	Echo request	Ping
11	Time exceeded	Traceroute (TTL reached 0)
13 / 14	Timestamp request / reply	-PP host discovery
17 / 18	Address mask request / reply	-PM host discovery

ARP ping scan

On the same local network, hosts must answer ARP or they cannot talk to anyone. A host firewall rarely blocks ARP. So the ARP ping (-PR) is the most reliable discovery method on a LAN. It works only on the local subnet because ARP does not cross routers. **ADDED CONTEXT:** Nmap uses ARP automatically for local targets, even if you pick another ping type (use --disable-arp-ping to stop that).

UDP ping scan

Nmap sends a UDP datagram to a port that is probably closed. A live host answers with an ICMP port unreachable message (type 3, code 3). That answer proves the host is up. **ADDED CONTEXT:** the default port is 40125 (verify).

TCP ping scans (SYN and ACK)

- **TCP SYN ping (-PS):** sends a SYN. Both SYN/ACK (port open) and RST (port closed) prove the host is alive. Default port 80 (ADDED CONTEXT).
- **TCP ACK ping (-PA):** sends an ACK that belongs to no connection. A live host answers RST. This can pass stateless firewalls that block SYN packets. Default port 80 (ADDED CONTEXT).

IP protocol ping scan

The probe is an IP packet that carries a chosen protocol number (for example 1 = ICMP, 2 = IGMP, 4 = IP-in-IP). The host is alive if it answers with the same protocol or with an ICMP "protocol unreachable" message. **ADDED CONTEXT:** the defaults are protocols 1, 2, and 4.

> **TRAP:** -sn does not mean "SYN". It means "ping scan, skip the port scan". The SYN ping is -PS and the SYN port scan is -sS. Do not mix them up.

ADDED CONTEXT: two related options. -Pn means "do not ping, assume every host is up". Use it when firewalls block pings and live hosts look dead. It is slow on large ranges. -sL (list scan) only prints the targets and resolves names without sending any probe. If you give no discovery option and run as root, Nmap sends an ICMP echo, a TCP

SYN to port 443, a TCP ACK to port 80, and an ICMP timestamp (the same as -PE -PS443 -PA80 -PP).

Ping sweep tools on the slide: Angry IP Scanner, SolarWinds Engineer's Toolset, NetScanTools Pro, Colasoft Ping Tool, Advanced IP Scanner, OpUtils. See section 4 for what each tool does.

Screenshots on the slide: two terminal captures on Parrot OS. The first, "TCP SYN Ping Scan", runs a SYN ping against 10.10.1.11. The second, "ICMP ECHO Ping Sweep", runs an echo ping across several addresses. The exact command lines are small [UNCLEAR on page 2]. The best readings are shown below.

Sample output 1 - single host (best reading of the TCP SYN ping capture):

```
$ sudo nmap -sn -PS 10.10.1.11
Starting Nmap 7.94 ( https://nmap.org ) at 2024-03-04 03:43 EST
Nmap scan report for 10.10.1.11
Host is up (0.00073s latency).
MAC Address: 00:15:5D:01:80:00 (Microsoft)
Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds
```

Line by line:

- Line 1: the command. sudo is needed because ping probes use raw packets.
- Line 2: Nmap version and start time.
- Line 3: the host that answered.
- Line 4: "Host is up" is the result. The number is the round-trip delay.
- Line 5: the MAC address was learned through ARP, so the target is on the same LAN. The prefix 00:15:5D belongs to Microsoft (it is used by Hyper-V virtual adapters, verify).
- Line 6: summary. "1 host up" is the count of live hosts.

Sample output 2 - ping sweep (illustrative):

```
$ sudo nmap -sn -PE 10.10.1.0/24
Nmap scan report for 10.10.1.1
Host is up (0.00061s latency).
MAC Address: 00:15:5D:01:80:10 (Microsoft)
Nmap scan report for 10.10.1.11
Host is up (0.00073s latency).
MAC Address: 00:15:5D:01:80:00 (Microsoft)
Nmap done: 256 IP addresses (2 hosts up) scanned in 2.15 seconds
```

The summary line shows how many addresses were scanned (256) and how many answered (2). No port information appears, because -sn stops after discovery.

Real-world use: A tester receives the range 10.10.1.0/24. A plain ICMP sweep finds only 2 hosts because the firewall drops echo requests. She repeats with -PS443 and -PA80 and finds 9 more. Result: a complete target list for port scanning.

Tools and commands used here: Nmap (-sn, -PE, -PP, -PM, -PR, -PU, -PS, -PA, -PO, -Pn), Zenmap, Angry IP Scanner, NetScanTools Pro, and the other ping sweep tools.

AI angle (ADDED CONTEXT): An attacker can ask an LLM to pick the probe type from the symptoms. Example prompt: "ICMP echo is blocked on 10.10.1.0/24. Give me an Nmap command that finds live hosts using TCP SYN to ports 80 and 443 and saves the live IPs to live.txt." A typical answer is sudo nmap -sn -PS80,443 10.10.1.0/24 -oG - followed by a filter. Read it before you run it.

Countermeasures and defense:

- Block inbound ICMP echo, timestamp, and address mask requests at the perimeter (see 3.19).
- Do not rely on ICMP blocking alone, because TCP, UDP, and ARP pings still work.
- Use an IDS to alert on many probes to sequential addresses.
- Keep hosts behind a stateful firewall that drops unsolicited ACK and SYN probes.

EXAM FOCUS:

- How it is asked: "Which Nmap option performs an ICMP timestamp ping?" -> -PP. "Which option performs an ARP ping scan?" -> -PR. "Which option disables port scanning after host discovery?" -> -sn.
- Keywords: "address mask" -> -PM. "Protocol number" -> -PO. "Local subnet, most reliable" -> ARP. "Firewall blocks ICMP" -> TCP SYN or ACK ping.
- Trap: a "ping sweep" is ICMP echo to a range (-PE with a range). It is not the same as -sn alone. -sn is the "no port scan" switch and can be used with any ping type.
- Look-alike: -PS (SYN ping, host discovery) vs -sS (SYN scan, port scanning). -PU (UDP ping) vs -sU (UDP port scan). -Pn (skip discovery) vs -sn (discovery only).

Memory trick: The P-flags in order: "Every Pinger Must Run Under Spotlights And Operate" -> -PE echo, -PP timestamp, -PM mask, -PR ARP, -PU UDP, -PS SYN, -PA ACK, -PO protocol.

3.6 Port Scanning Techniques

What it is (simple words): Port scanning finds out which ports on a live host are open, closed, or filtered. The slide groups the techniques by the type of protocol used for communication: TCP, UDP, SCTP, SSDP, and IPv6. The goal is a list of open ports, because every open port is a running service that may have a weakness.

- **Open port:** a service is listening and accepts connections.
- **Closed port:** the host is reachable, but nothing listens on that port.
- **Filtered port:** a firewall or filter blocks the probe, so the scanner cannot tell if the port is open.
- **Stealth scan:** a scan that avoids completing a full TCP connection, so the target application does not log it.

Analogy: You have a list of occupied houses (host discovery). Now you walk to each house and test every door and window. An open door lets you in. A closed door says "nobody is here for this service". A door behind a guard who ignores you is filtered.

How it works (numbered steps):

1. Take the live hosts from host discovery.
2. Choose a scan type (connect, SYN, Xmas, ACK, UDP, and so on).
3. Send probe packets to each port in a list (Nmap scans the 1000 most common ports by default).
4. Read the reply for each port. The reply, or the lack of one, gives the port state.
5. Report open ports with the service name Nmap expects on that port.

ADDED CONTEXT: the six port states in Nmap.

State	Meaning	Typical cause
open	A service accepts connections	Service running
closed	Reachable, nothing listening	Host answers RST
filtered	Probe blocked, state unknown	Firewall drops or rejects
unfiltered	Reachable, open or closed unknown	ACK scan result only
open or filtered	No reply, cannot decide	UDP, FIN, NULL, Xmas, SCTP cookie
closed or filtered	Cannot decide	Idle scan only (rare)

ADDED CONTEXT: port ranges. Well-known ports are 0 to 1023 (system services). Registered ports are 1024 to 49151. Dynamic or private ports are 49152 to 65535. There are 65536 ports (0 to 65535) per protocol (TCP and UDP are separate).

The technique tree from the slide:

```
PORT AND SERVICE DISCOVERY TECHNIQUES
|
+-- TCP Scanning
|   +-- Open TCP scanning methods
|   |   +-- TCP Connect / Full Open Scan   nmap -sT -v
|   +-- Stealth TCP scanning methods
|   |   +-- Half-open Scan                 nmap -sS -v
|   |   +-- Inverse TCP Flag Scan          nmap -sF -sN -sX -v
|   |   |   +-- Xmas Scan                  nmap -sX -v
|   |   |   +-- FIN Scan                   nmap -sF -v
|   |   |   +-- NULL Scan                  nmap -sN -v
|   |   |   +-- Maimon Scan                nmap -sM -v
|   |   +-- ACK Flag Probe Scan            nmap -sA -v
|   |       +-- TTL-based Scan             nmap -sA --ttl 100 -v
|   |       +-- Window Scan                nmap -sW -v
|   +-- Third party and spoofed methods
|       +-- IDLE / IPID Header Scan        nmap -Pn -p- -sI ZOMBIE TARGET
+-- UDP Scanning                           nmap -sU -v
+-- SCTP Scanning
|   +-- SCTP INIT Scanning                 nmap -sY -v
|   +-- SCTP COOKIE ECHO Scanning          nmap -sZ -v
+-- SSDP Scanning
+-- IPv6 Scanning                          nmap -6 TARGET
```

[UNCLEAR on page 3]: the thin connector lines make the exact parent of Maimon, TTL-based, and Window scans hard to see. The grouping above is the best reading and matches the usual EC-Council layout. The slide prints the TTL-based command as -sA -ttl 100 and the window scan as -sA -sW; the real Nmap options are --ttl and -sW (verify against your version).

Summary of what each scan sends and gets back (TCP and others):

Scan	Flag	Probe sent	Open port reply	Closed port reply
TCP connect	-sT	SYN, then full handshake	SYN/ACK	RST
SYN (half-open)	-sS	SYN	SYN/ACK	RST
Xmas	-sX	FIN + PSH + URG	No reply	RST
FIN	-sF	FIN	No reply	RST
NULL	-sN	No flags	No reply	RST
Maimon	-sM	FIN + ACK	No reply (BSD drops it)	RST
ACK	-sA	ACK	RST (unfiltered)	RST (unfiltered)
Window	-sW	ACK	RST with window above 0	RST with window 0
UDP	-sU	Empty UDP datagram	UDP reply or no reply	ICMP port unreachable
SCTP INIT	-sY	INIT chunk	INIT-ACK	ABORT
SCTP COOKIE ECHO	-sZ	COOKIE ECHO chunk	No reply	ABORT

Scan	Flag	Probe sent	Open port reply	Closed port reply
Idle	-sI	Spoofed SYN via zombie	Zombie IP ID rises by 2	Zombie IP ID rises by 1

In every TCP row, no reply at all (or an ICMP unreachable error) from a SYN-type probe means the port is filtered.

TCP connect scan, also full open scan (-sT)

What it is: The scanner asks the operating system to open a real TCP connection to each port. It completes the full three-way handshake (SYN, SYN/ACK, ACK).

How it works:

1. The scanner sends SYN to the port.
2. If the port is open, the host answers SYN/ACK. The scanner sends ACK, so the connection exists, and then closes it.
3. If the port is closed, the host answers RST.
4. If nothing comes back, the port is filtered.

Why use it: It needs no special privileges (no root), because it uses the normal connect() call. It is the default Nmap scan for a user without raw socket rights.

Downsides: It is slow and noisy. The target service sees a real connection and logs it. An IDS spots many connections from one source quickly.

Command and reading:

```
nmap -sT -v 10.10.1.11
```

In the module's MAC spoofing screenshot, a connect scan shows "Not shown: 994 closed tcp ports (conn-refused)". The reason text "conn-refused" is typical of a connect scan. A SYN scan shows "reset" instead.

SYN scan, also half-open or stealth scan (-sS)

What it is: The scanner sends SYN and watches the reply. It never finishes the handshake. After SYN/ACK it sends RST, so the connection is never established. This is why it is called half-open.

How it works:

1. The scanner sends SYN.
2. SYN/ACK back = open. The scanner then sends RST to tear it down.
3. RST back = closed.
4. No reply or ICMP unreachable = filtered.

Why use it: It is fast and, because no connection completes, many applications do not log it. It is the default scan when you run Nmap as root or administrator.

Downsides: It needs raw socket access (root). Modern firewalls and IDS products still detect and log SYN scans, so "stealth" is a historic name and not a guarantee.

```
sudo nmap -sS -v 10.10.1.11
```

Sample output (based on the module's screenshots, shortened):

```
Nmap scan report for 10.10.1.11
Host is up (0.00042s latency).
Not shown: 994 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
80/tcp   open  http
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
3389/tcp open  ms-wbt-server
MAC Address: 00:15:5D:01:80:00 (Microsoft)
Nmap done: 1 IP address (1 host up) scanned in 1.65 seconds
```

Line by line:

- "Not shown: 994 closed tcp ports (reset)": Nmap scanned 1000 ports. 994 were closed and replied RST, so it hides them to keep the report short. "reset" is the reason.
- The table lists the six open ports with the service name that Nmap expects on each port number.
- 21 = FTP, 80 = HTTP, 135 = Microsoft RPC, 139 = NetBIOS session, 445 = SMB (microsoft-ds), 3389 = RDP (ms-wbt-server). This set of ports points to a Windows machine.
- "scanned in 1.65 seconds": a local SYN scan is fast.

> **EXAM TIP:** Connect scan = full handshake, no root needed, logged. SYN scan = half-open, needs root, quieter. If the question says "does not complete the TCP handshake", the answer is SYN (half-open).

Inverse TCP flag scanning: Xmas, FIN, and NULL (-sX, -sF, -sN)

What it is: The scanner sends a TCP probe with unusual flags. The slide calls the group "Inverse TCP Flag Scan". The logic is inverted: an open port stays silent, and a closed port answers.

- **Xmas scan (-sX):** sets FIN, PSH, and URG. The packet is "lit up like a Christmas tree".
- **FIN scan (-sF):** sets only FIN.
- **NULL scan (-sN):** sets no flags at all.

How it works: RFC 793 (the TCP standard) says a closed port must answer an unexpected packet with RST, and an open port must silently ignore it.

1. The scanner sends the odd probe.
2. No response = the port is open or filtered ("open or filtered").
3. RST = the port is closed.
4. ICMP unreachable (type 3) = the port is filtered.

Why use them: They can slip through simple (non-stateful) packet filters that only block SYN packets, and they are a little quieter than a SYN scan.

Big limits:

- They only work on systems that follow RFC 793 closely (most Unix and Linux). Windows, many Cisco devices, BSDI, and IBM OS/400 answer RST on every port, so every port looks closed.
- They cannot separate open ports from filtered ones (the result is "open or filtered").
- Modern IDS products detect them.

```
sudo nmap -sX -v 10.10.1.20
sudo nmap -sF -v 10.10.1.20
sudo nmap -sN -v 10.10.1.20
```

Sample output, Linux target (illustrative):

```
Nmap scan report for 10.10.1.20
Host is up (0.00040s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE         SERVICE
22/tcp open|filtered ssh
80/tcp open|filtered http
```

How to read it: the two ports that did not answer RST are shown as open|filtered. A tester must confirm them with another scan (for example -sV or a connect scan).

Sample output, Windows target (illustrative):

```
Nmap scan report for 10.10.1.11
Host is up (0.00031s latency).
All 1000 scanned ports on 10.10.1.11 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
```

How to read it: Windows replied RST to everything, so Nmap reports all ports closed. This does not mean the host has no services. It means these scan types do not work on Windows.

> **TRAP:** "No response" means open for FIN, NULL, and Xmas scans. For a SYN scan, "no response" means filtered. Read the scan type before you read the result.

Maimon scan (-sM)

What it is: A variation of FIN, NULL, and Xmas scans. The probe has the FIN and ACK flags. It is named after Uriel Maimon, who described it in Phrack magazine in 1996.

How it works: By the standard, a closed or open port should reply RST. Many BSD-derived systems drop the packet when the port is open, and reply RST when it is closed. So, as with the other inverse flag scans, silence means open (or filtered) and RST means closed.

```
sudo nmap -sM -v 10.10.1.20
```

> **EXAM TIP:** Maimon probe = FIN + ACK. Xmas probe = FIN + PSH + URG. FIN probe = FIN only. NULL probe = no flags.

ACK flag probe scan (-sA)

What it is: The scanner sends a packet with only the ACK flag set, which belongs to no connection. It does not find open ports. It maps the firewall rules: which ports are filtered and which are not.

How it works:

1. The scanner sends an ACK to the port.
2. RST back = the packet reached the host. The port is "unfiltered" (it may be open or closed, the scan cannot tell).
3. No reply, or an ICMP unreachable error = a firewall blocks the port. The port is "filtered".

Why use it: To learn whether a firewall is stateful. A stateful firewall drops stray ACK packets. A simple packet filter lets them through.

```
sudo nmap -sA -v 10.10.1.11
```

Sample output (illustrative):

```
Not shown: 996 filtered tcp ports (no-response)
PORT    STATE      SERVICE
80/tcp  unfiltered http
443/tcp unfiltered https
```

How to read it: ports 80 and 443 pass through the firewall (unfiltered). Everything else is blocked (filtered). Nothing here says whether 80 is open.

> **TRAP:** ACK scan never says "open". If a question asks which scan finds open ports, ACK is wrong. If it asks which scan maps firewall rules, ACK is right.

TTL-based scan (ACK probe with a set TTL)

What it is: A variation of the ACK flag probe scan. The slide lists it under ACK scanning, with the command nmap -sA -ttl 100 -v. The real Nmap option is --ttl (verify), which sets the TTL (time to live) of the packets Nmap sends.

How it works (EC-Council explanation, verify): The scanner sends ACK probes and looks at the TTL of the RST replies. A small TTL in the reply (below a boundary of 64) suggests an open port. A larger TTL suggests a closed port.

```
sudo nmap -sA --ttl 100 -v 10.10.1.11
```

> **ADDED CONTEXT:** The TTL-based and window-based ideas are courseware techniques. Nmap's own manual describes the ACK scan and the window scan, not a TTL scan, so treat the TTL rule as a theory fact for the exam and verify it before relying on it in a real test.

TCP window scan (-sW)

What it is: Like the ACK scan, but it reads the TCP window field of the RST replies. On some systems an open port returns a RST with a positive window size and a closed port returns a RST with window size 0.

How it works:

1. The scanner sends an ACK probe.
2. RST with window above 0 = open.
3. RST with window 0 = closed.
4. No reply or ICMP unreachable = filtered.

```
sudo nmap -sW -v 10.10.1.11
```

Limit: It depends on an implementation detail of a minority of systems. On systems that do not behave this way, all ports look closed. Do not trust it alone.

> **TRAP:** The slide prints the window scan as "nmap -sA -sW -v". The scan flag that matters is -sW. Remember: -sA = ACK scan, -sW = window scan.

Idle scan, also IDLE/IPID header scan (-sI)

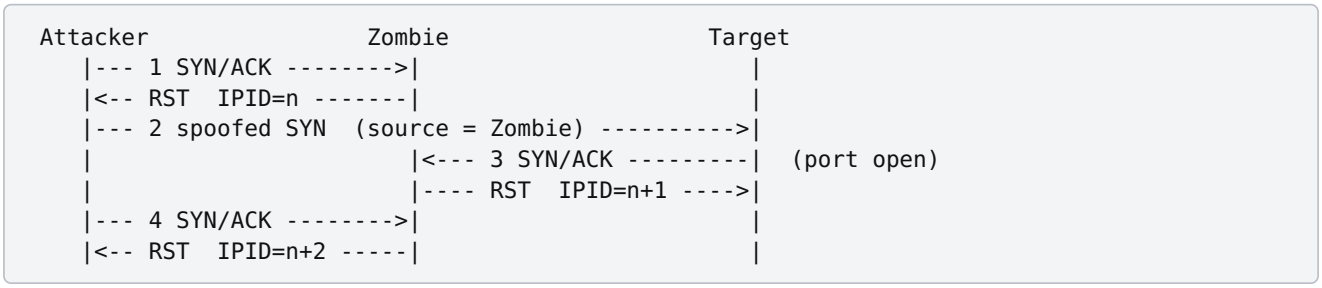
What it is: A third-party, spoofed scan. The scanner never sends a packet to the target from its own IP address. Instead, it borrows an innocent "zombie" host, and reads the

zombie's IP ID counter (the IPID header field) to learn what the target answered. The target sees the zombie as the scanner.

- **Zombie:** an idle host with a predictable, incrementing IP ID. Older Windows hosts, some printers and embedded devices fit. Most modern systems randomize the IP ID, so they are poor zombies.
- **IP ID (IPID):** a 16-bit number in the IP header that identifies a packet, used for fragment reassembly. Some hosts add 1 for every packet they send.

How it works (numbered steps):

1. The attacker sends a SYN/ACK to the zombie. The zombie replies RST with IP ID = n. The attacker records n.
2. The attacker sends a SYN to the target with the zombie's address as the spoofed source.
3. If the target's port is open, the target sends SYN/ACK to the zombie. The zombie did not start a connection, so it replies RST. This RST uses the next IP ID (n+1).
4. If the port is closed, the target sends RST to the zombie. The zombie ignores a RST and sends nothing. The IP ID does not change.
5. The attacker probes the zombie again. The reply has IP ID = n+2 if the port was open, or n+1 if it was closed or filtered.



Rule: IP ID rises by 2 = port open. IP ID rises by 1 = port closed or filtered.

Command from the slide:

```
sudo nmap -Pn -p- -sI ZOMBIE_IP TARGET_IP
```

- -Pn: skip host discovery, because pinging the target from your own IP would reveal you.
- -p-: scan all 65535 ports.
- -sI ZOMBIE_IP: use the idle scan with this zombie (optionally ZOMBIE_IP:PROBE_PORT).
- TARGET_IP: the host to scan.

Why use it: It is a blind scan. The target's logs show the zombie's IP, not the attacker's. It can also reveal trust relationships (rules that allow the zombie but block the attacker).

Limits: It needs a suitable zombie, it is slow, and the network must allow spoofed packets to leave (no egress filtering).

> **EXAM TIP:** Keywords "zombie", "IPID", "blind scan", "attacker's IP never seen by the target" -> idle scan, nmap -sI.

UDP scan (-sU)

What it is: UDP (User Datagram Protocol) has no handshake and no connection. The scanner sends a UDP datagram and watches for a reply.

How it works:

1. The scanner sends an empty (or protocol-specific) UDP datagram to the port.
2. ICMP port unreachable (type 3, code 3) = closed.
3. A UDP reply = open.
4. No reply = open or filtered (the scanner cannot tell).
5. Other ICMP unreachable errors = filtered.

Why it is slow: Operating systems limit the rate of ICMP errors (Linux about one per second), so scanning 65535 UDP ports can take hours. Scan the common UDP ports only (53, 67, 69, 123, 137, 161, 500, 1900) or use --top-ports.

```
sudo nmap -sU -v 10.10.1.11
sudo nmap -sU -p 53,161 10.10.1.11
```

Sample output (illustrative):

```
PORT    STATE         SERVICE
53/udp  open          domain
123/udp open|filtered ntp
161/udp open|filtered snmp
```

> **TRAP:** UDP scanning needs the UDP-specific flag -sU. A TCP scan (-sS) does not test UDP services such as DNS, SNMP, and TFTP. Many real breaches start on an ignored UDP port.

SCTP scans: INIT (-sY) and COOKIE ECHO (-sZ)

What it is: SCTP (Stream Control Transmission Protocol) is a transport protocol (IP protocol number 132) used for telephony signalling and some mobile network interfaces. It sets up an association with a four-way handshake: INIT, INIT-ACK, COOKIE-ECHO, COOKIE-ACK. The word "chunk" means one message part inside an SCTP packet.

- **SCTP INIT scan (-sY):** the SCTP twin of the SYN scan. It sends an INIT chunk and never finishes the association.
- INIT-ACK back = open.

- ABORT back = closed.
- No reply or ICMP unreachable = filtered.
- **SCTP COOKIE ECHO scan (-sZ):** sends a COOKIE ECHO chunk.
- Open port: silently drops it (no reply) -> open or filtered.
- Closed port: replies ABORT.
- It can pass simple filters that block only INIT chunks, but it cannot separate open from filtered.

```
sudo nmap -sY -v 10.10.1.11
sudo nmap -sZ -v 10.10.1.11
```

> **EXAM TIP:** SCTP INIT = -sY (like SYN). SCTP COOKIE ECHO = -sZ (like the FIN or NULL scans, silence means open or filtered).

SSDP scanning

What it is: SSDP (Simple Service Discovery Protocol) is the discovery part of UPnP (Universal Plug and Play). Devices such as routers, cameras, smart TVs, and printers announce themselves over UDP port 1900. The multicast address is 239.255.255.250. The slide lists SSDP scanning in the tree but gives no command (ADDED CONTEXT below).

How it works:

1. The scanner sends an M-SEARCH request to UDP 1900.
2. UPnP devices reply with their type, location URL, and server string.
3. The attacker learns which devices exist and looks for UPnP flaws.

```
M-SEARCH * HTTP/1.1
HOST: 239.255.255.250:1900
MAN: "ssdp:discover"
MX: 2
ST: ssdp:all
```

ADDED CONTEXT: commands (verify script and module names).

```
sudo nmap -sU -p 1900 --script upnp-info 10.10.1.0/24
```

In Metasploit, the module auxiliary/scanner/upnp/ssdp_msearch sends the same M-SEARCH request.

Why attackers care: Exposed UPnP can let an attacker open ports on a router, and SSDP can be abused for reflection and amplification denial-of-service (Module 10).

Defense: Disable UPnP on routers, block UDP 1900 at the perimeter, and keep device firmware current.

IPv6 scanning (-6)

What it is: Scanning hosts that use IPv6 addresses (128-bit, for example 2001:db8::10). The slide gives nmap -6 followed by the target IP or domain.

How it works:

1. Add -6 so Nmap uses IPv6.
2. Give a single IPv6 address or a domain name that has an AAAA record.
3. Nmap runs the chosen scan over IPv6.

```
nmap -6 2001:db8::10
nmap -6 -sS -p 22,80,443 www.example.com
```

ADDED CONTEXT: IPv6 subnets are so large that a ping sweep of a /64 is not practical. Attackers find IPv6 hosts through DNS (AAAA records), local multicast (ping to ff02::1, the all-nodes address), or leaked logs. IPv6 is often less filtered than IPv4. ICMPv6 has no timestamp query, and Neighbor Discovery replaces ARP.

Real-world use (for the whole of 3.6): A tester finds 4 live hosts. A SYN scan lists the open ports on each. An ACK scan against the firewall shows that only ports 80 and 443 pass. A UDP scan of the top ports finds an open SNMP service on port 161. Result: three exposures for the report.

Tools and commands used here: Nmap and Zenmap (all scans), Hping3 (custom TCP/UDP probes), Metasploit (portscan modules), NetScanTools Pro.

AI angle (ADDED CONTEXT): An LLM can translate "scan the top 100 UDP ports quietly and save XML" into sudo nmap -sU --top-ports 100 -T2 -oX out.xml. It can also summarize a long scan report. Example prompt: "Explain why this Nmap output shows open|filtered for every port and what scan I should try next." Check each flag against section 5.

Countermeasures and defense:

- Stateful firewall plus IDS rules that detect SYN, FIN, NULL, Xmas, and ACK scans.
- Block unused ports and run only needed services.
- Rate-limit and log ICMP unreachable messages and RST packets.
- Run your own scans to check the rules (see 3.19).

EXAM FOCUS:

- How it is asked: "Which scan sets the FIN, PSH, and URG flags?" -> Xmas. "Which scan does not complete the three-way handshake?" -> SYN (half-open). "Which scan uses a zombie?" -> idle. "Which scan maps firewall rules?" -> ACK.
- Keywords: "RFC 793", "no response means open" -> FIN, NULL, Xmas. "Windows answers RST to everything" -> these scans fail on Windows.

- Trap: for UDP, a closed port gives ICMP port unreachable, and no reply means open or filtered. For TCP SYN, no reply means filtered. Keep the two straight.
- Look-alike: -sS (TCP SYN) vs -sY (SCTP INIT). -sF (FIN) vs -sM (FIN+ACK). -sA (ACK) vs -sW (window).

Memory trick: Flag letters: "T for Three-way (connect), S for Syn, X for Xmas, F for Fin, N for Null, M for Maimon, A for Ack, W for Window, I for Idle, U for Udp, Y and Z for SCTP (the last letters in the alphabet)."

3.7 Service Version Discovery

What it is (simple words): Service version detection helps attackers obtain information about the running services and their versions on a target system. Knowing the exact version number lets an attacker decide whether the system is vulnerable to a particular exploit. In Zenmap (the Nmap graphical front end), the -sV option detects service versions.

- **Service version:** the product name and release of the program behind a port, for example "Microsoft IIS httpd 10.0".
- **Exploit:** code that takes advantage of a known weakness in a specific software version.
- **Zenmap:** the official graphical interface for Nmap. It runs the same scans and shows the results in tabs.

Analogy: An open port tells you there is a shop at address 80. Version detection reads the sign above the door: brand, model, and year. With the model number you can look up its known defects.

How it works (numbered steps):

1. A port scan finds the open ports.
2. For each open port, Nmap sends probes and reads the greeting banner or the answer to a protocol request.
3. Nmap compares the answer to its database of service signatures (nmap-service-probes).
4. It prints the product, version, and sometimes OS and host name details.
5. The attacker searches the version in a vulnerability database (Module 05) to find exploits.

The screenshot (Zenmap with target 10.10.1.11 and command nmap -sV 10.10.1.11):

```
Starting Nmap 7.94 ( https://nmap.org ) at 2024-03-11 23:35
Nmap scan report for 10.10.1.11
Host is up (0.0011s latency).
Not shown: 994 closed tcp ports (reset)
PORT     STATE SERVICE       VERSION
21/tcp   open  ftp           Microsoft ftpd
80/tcp   open  http          Microsoft IIS httpd 10.0
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp  open  microsoft-ds  Microsoft Windows 7 - 10 microsoft-ds
                             (workgroup: WORKGROUP)
3389/tcp open  ms-wbt-server Microsoft Terminal Services
MAC Address: 00:15:5D:01:80:00 (Microsoft)
Service Info: Host: WINDOWS11; OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 46.55 seconds
```

Some version strings and the date are small in the screenshot [UNCLEAR on page 3]; the values above are the best reading.

Line by line:

- The new VERSION column is what -sV adds. Compare with the plain SYN scan, which only shows the service name.
- "Microsoft IIS httpd 10.0" tells the attacker the web server is IIS 10.0, so the OS is Windows 10, 11, or Server 2016 or later.
- "Service Info: Host ... OS: Windows; CPE ..." is a summary of what Nmap learned. CPE (Common Platform Enumeration) is a standard name format for products, here the Microsoft Windows OS.
- The scan took 46.55 seconds instead of 1.65 seconds, because Nmap sent extra probes to each open port. Version detection is slower and noisier than a plain port scan.

ADDED CONTEXT: related options.

Option	Meaning
-sV	Probe open ports to find service and version
--version-intensity 0-9	How hard to probe (default 7)
--version-light / --version-all	Intensity 2 / intensity 9
-O	Guess the OS (see 3.8, 3.13)
-A	Aggressive: -sV, -O, -sC scripts, and traceroute

Real-world use: A scan shows port 21 open. Version detection says "vsftpd 2.3.4". The tester knows this release contained a backdoor and reports a critical finding at once.

Tools and commands used here: Nmap -sV, Zenmap. Related: Netcat and Telnet (banner grabbing, section 5).

AI angle (ADDED CONTEXT): Paste the VERSION column into an LLM and ask: "List the known CVEs for each of these versions and rank them by exploitability." Treat the answer as a lead, not proof. LLMs can invent CVE numbers, so confirm each one in the NVD (National Vulnerability Database).

Countermeasures and defense: Remove or edit version banners (for example, ServerTokens Prod in Apache, remove the IIS Server header), patch services, and block unused ports.

EXAM FOCUS:

- How it is asked: "Which Nmap option detects the versions of running services?" -> -sV. "Why do attackers detect service versions?" -> to find vulnerabilities and exploits for those versions.
- Keywords: "VERSION column", "Zenmap" -> -sV.
- Trap: -sV is not -sS. -sV needs open ports first and then adds probes. -O is for the OS. -A includes both.

Memory trick: "-sV = Service Version. -O = Operating system."

3.8 How to Identify Target System OS

What it is (simple words): Attackers can identify the OS running on the target machine by looking at the Time To Live (TTL) and the TCP window size in the IP header of the first packet in a TCP session. This is a passive way to guess the OS, because it only reads values the target already sends, with no extra probes beyond a normal connection.

- **TTL (Time To Live):** a counter in the IP header that drops by 1 at every router. Different operating systems start it at different default values.
- **TCP window size:** the amount of data (in bytes) a host says it can accept before it must be acknowledged. Operating systems set different default window sizes.
- **Packet sniffing:** capturing network traffic as it passes, without sending anything new to the target.

Analogy: Different car brands drive off the factory line with different default radio volume and mirror angle settings. If you measure those two settings on a passing car, you can guess the brand even without reading the badge.

How it works (numbered steps):

1. The attacker captures (sniffs) the response generated from the target machine using a packet-sniffing tool such as Wireshark.
2. He looks at the first packet of a TCP session from the target (usually the SYN/ACK reply).
3. He reads the TTL field and the TCP window size field in that packet.
4. He compares both values against a reference table of known OS defaults.

5. He picks the OS whose defaults match most closely. This is a guess, not a certainty, because TTL drops with every hop and some systems let administrators change the defaults.

Window size values for OS (from the slide):

Operating system	Time to live	TCP window size
Linux	64	5840
FreeBSD	64	65535
OpenBSD	255	16384
Windows	128	65,535 bytes to 1 Gigabyte
Cisco Routers	255	4128
Solaris	255	8760
AIX	255	16384

The Windows window size entry is wide ("65,535 bytes to 1 Gigabyte") because modern Windows versions use TCP window scaling, which lets the effective window grow far beyond the 16-bit header field through a scaling factor (ADDED CONTEXT).

Screenshot on the slide: two Wireshark captures side by side. The left one is labeled "Possible OS is Windows" and the right one "Possible OS is Linux". The highlighted fields in each capture are the TTL and window size inside the IP and TCP headers of a captured packet. The exact byte values shown in each capture are too small to read with confidence [UNCLEAR on page 4]; the reference table above gives the values to compare against.

ADDED CONTEXT: how to read it in Wireshark. Open a capture, click a SYN or SYN/ACK packet from the target, expand "Internet Protocol Version 4" to find "Time to live", and expand "Transmission Control Protocol" to find "Window size value" (and "Calculated window size" if scaling is used). Compare both numbers to the table.

```
Internet Protocol Version 4, Src: 10.10.1.11
    Time to live: 128              <-- matches Windows
Transmission Control Protocol, Src Port: 80
    Window size value: 8192
    [Calculated window size: 65535]
```

A TTL of 128 strongly suggests Windows. A TTL of 64 suggests Linux or FreeBSD; the window size then tells them apart (5840 leans Linux, 65535 leans FreeBSD).

> **TRAP:** The TTL you observe is the target's starting TTL minus the number of router hops in between, not the raw starting value. A captured TTL of 118 from a Windows host 10 hops away still points to a 128 start (118 is close to 128, and router hops subtract small whole numbers). If the captured value does not match any row closely, assume several hops have already reduced it and compare the "closest higher" default instead of expecting an exact match.

Real-world use: A tester captures a reply with TTL 128 and window size 8192. She concludes the target is Windows and chooses Windows-specific enumeration tools and exploits for the next phase.

Tools and commands used here: Wireshark (capture and inspect), tcpdump (command-line capture), Nmap -O (active OS fingerprinting, ADDED CONTEXT, covered in 3.13 alongside banner grabbing).

AI angle (ADDED CONTEXT): An LLM can read a short list of TTL and window values and suggest the most likely OS, citing the same kind of reference table. Example prompt: "A captured packet has TTL 117 and TCP window 8192. Which OS is the host most likely running, and why?" Treat the answer as a lead to confirm with a second method (banner grabbing or -sV).

Countermeasures and defense: Normalize or randomize TTL and window size at the firewall or load balancer so OS fingerprints do not leak; keep systems patched regardless of what an attacker guesses.

EXAM FOCUS:

- How it is asked: "A captured packet shows TTL 255 and a small window size. Which operating systems are the most likely candidates?" -> Cisco, Solaris, OpenBSD, or AIX (all use TTL 255; the window size picks among them).
- Keywords: "TTL and TCP window size", "first packet in a TCP session" -> this passive OS-guessing technique.
- Trap: TTL 64 fits both Linux and FreeBSD. You need the window size (5840 vs 65535) to separate them.
- Look-alike: this passive method (reading TTL and window from a normal reply) vs active banner grabbing (section 3.13), which sends crafted packets on purpose to provoke a telling response.

Memory trick: "Windows is 128, everyone big (255) looks alike, Linux and FreeBSD share 64." -> Windows=128; {Cisco, Solaris, OpenBSD, AIX}=255; {Linux, FreeBSD} =64.

3.9 Packet Fragmentation

What it is (simple words): Packet fragmentation refers to the splitting of a probe packet into several smaller packets (fragments) while sending it to a network. It is not a

new scanning method; it is a modification of the previous scanning techniques (SYN, FIN, and the rest). The TCP header itself is split across several IP fragments, so simple packet filters cannot see the complete header and cannot tell what the packets are intended to do.

- **IP fragment:** a piece of an original IP packet. The receiving host's IP stack reassembles the fragments before handing the data up to TCP.
- **Packet filter:** a simple firewall that checks fields in a packet header against rules. Many older or simpler filters inspect only the first fragment, or cannot reassemble fragments at all.

Analogy: Instead of mailing one envelope that clearly says "invoice", you tear the letter into three strips and mail each strip separately. A guard who only reads whole envelopes at the gate cannot tell what the strips add up to until they are taped back together on the other side.

How it works (numbered steps):

1. The attacker chooses a normal scan type, for example a SYN or FIN scan.
2. Instead of sending each probe as one full packet, the tool splits the IP packet into small fragments (commonly 8 or 16 bytes of data per fragment, with an offset field that tells the receiver how to reassemble them).
3. A simple, non-stateful firewall or IDS sees only tiny, incomplete pieces and may pass them through without matching any rule.
4. The target's own IP stack reassembles the fragments into the original TCP header and processes the probe normally.
5. The target answers the probe as it would for any unfragmented packet: RST if closed, SYN/ACK if open (for a SYN probe), and so on, following the same reply logic as the base scan type.

SYN/FIN scanning using IP fragments (diagram on the slide):

```
   Attacker                                    Target
      |                                          |
      |-- SYN/FIN (small IP fragments) + port n ->|
      |                                          |
      |<---- RST (if port is closed) -------------|
```

The diagram shows the attacker sending a SYN or FIN probe to port n, broken into small IP fragments. If the port is closed, the target reassembles the fragments and answers RST, exactly as an unfragmented SYN or FIN scan would.

Command (from the Zenmap screenshot, best reading [UNCLEAR on page 4]; standard Nmap syntax below):

```
sudo nmap -f -v 10.10.1.11
sudo nmap -f -sS -v 10.10.1.11
sudo nmap --mtu 24 -sS -v 10.10.1.11
```

- -f: fragment packets (8 bytes of data per fragment after the header).
- -f -f (or -ff): use smaller fragments still (16-byte offsets, verify).
- --mtu NUMBER: set a custom fragment size, must be a multiple of 8.

The Zenmap screenshot on the slide shows a scan result with a fragmentation-related warning banner at the top of the output panel; the exact warning text is too small to read reliably [UNCLEAR on page 4]. A typical Nmap warning in this situation reads: "WARNING: Fragmentation (-f or --mtu) breaks OS detection and version scanning. It will likely cause issues with other scan types too."

Sample output (illustrative):

```
Starting Nmap 7.94 ( https://nmap.org ) at 2024-03-12 00:45
WARNING: Fragmentation (-f) breaks OS detection and version
scanning.
Nmap scan report for 10.10.1.11
Host is up (0.00061s latency).
Not shown: 994 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
80/tcp   open  http
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
3389/tcp open  ms-wbt-server
Nmap done: 1 IP address (1 host up) scanned in 20.14 seconds
```

How to read it: the result looks the same as a normal SYN scan (same six open ports), but the scan took much longer because each probe is now several small packets instead of one, and the warning confirms that fragmentation was active and may interfere with other detection features.

Real-world use: A tester finds that a plain SYN scan against a segment returns nothing (every port filtered), because a simple packet filter drops anything that looks like a scan signature. A fragmented SYN scan (-f) against the same segment gets through, because the filter only inspects whole, unfragmented packets.

Tools and commands used here: Nmap (-f, --mtu), Colasoft Packet Builder and NetScanTools Pro (manual fragment crafting, see 3.12), Hping3 (--frag, ADDED CONTEXT).

AI angle (ADDED CONTEXT): Ask an LLM: "My SYN scan against 10.10.1.0/24 returns all ports filtered, but the hosts answer ping. Suggest Nmap options that might get through a simple packet filter, and explain the risk of fragmentation to other features of the scan." A reasonable answer includes -f and a note that it conflicts with -O and -sV.

Fragmentation can also crash poorly written IDS or firewall software, so only use it in an authorized test.

Countermeasures and defense: Use a stateful firewall or IDS that reassembles fragments before inspecting them (most modern firewalls do this by default); drop or log unusual tiny fragments at the perimeter; keep firewall and IDS firmware current (also listed in 3.19).

EXAM FOCUS:

- How it is asked: "Which evasion technique splits a probe packet into several smaller packets so a packet filter cannot see the complete header?" -> packet fragmentation.
- Keywords: "split the TCP header across packets", "not a new scan, a modification" -> fragmentation.
- Trap: fragmentation is not its own scan type. It is applied on top of an existing scan (SYN, FIN, and so on). A question that asks "which scan type is this" when fragmentation is mentioned should still point to the underlying scan (SYN, FIN) plus the evasion flag.
- Look-alike: packet fragmentation (splits one packet into pieces) vs source port manipulation (3.10, changes which port the packet appears to come from) vs IP decoy (3.16, adds fake source IPs). All three are evasion techniques from the same list (3.14), but they change different things.

Memory trick: "Fragment it like a torn letter: the guard cannot read a torn strip, but the mailbox tapes it back together."

3.10 Source Port Manipulation

What it is (simple words): Source port manipulation refers to manipulating the actual port number used by the scanner with a common, trusted port number, in order to evade an IDS or firewall. It works because a firewall is often configured to allow packets that come from well-known ports, such as HTTP (80), DNS (53), or FTP (21), on the assumption that only a real server on that port would use it as a source.

- **Source port:** the port number a sender puts in its own outgoing packet, normally a random high (ephemeral) port chosen by the OS.
- **Trusted port rule:** a firewall rule that allows traffic because it appears to come from a well-known service port, rather than checking what the traffic actually is.

Analogy: A guard at a gate waves through any van that has "Electric Company" painted on the side, without checking who is driving or what is inside. An attacker paints the same logo on his own van (uses port 80 as his source) and drives straight through.

How it works (numbered steps):

1. The attacker identifies a well-known port the firewall is likely to trust as a source, such as 80, 53, or 20/21.
2. He sets his scanner's source port to that number instead of a random high port.
3. The firewall checks the packet's source port, sees a trusted number, and allows the packet through to the target.
4. The target processes the probe normally and replies according to the scan type being used (SYN, ACK, and so on).

Diagram on the slide:

```
                    +-----------+
  Actual Port 242    | Firewall  |      Port 242 BLOCKED (X)
     Attacker ------>|           |
  Manipulated Port 80 | allows   |---> Port 80 ALLOWED -> Victim
                      | port 80  |
                    +-----------+
```

The firewall is configured to allow manipulated port 80 traffic through to the victim, while a probe that honestly uses port 242 as its source is blocked.

Command (Nmap uses -g or --source-port):

```
sudo nmap -g 80 -sS -v 10.10.1.11
sudo nmap --source-port 53 -sU -v 10.10.1.11
```

- -g PORT or --source-port PORT: force Nmap to send every probe from the given source port number.
- Common choices: 20 (FTP data), 53 (DNS), 67 (DHCP server), 88 (Kerberos), 80 (HTTP), 443 (HTTPS).

Sample output (as seen in the module's screenshot, shortened):

```
Starting Nmap 7.94 ( https://nmap.org ) at 2024-03-12 00:53
Nmap scan report for 10.10.1.11
Host is up (0.0004s latency).
Not shown: 994 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
80/tcp   open  http
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
3389/tcp open  ms-wbt-server
MAC Address: 00:15:5D:01:00:00 (Microsoft)
Nmap done: 1 IP address (1 host up) scanned in 1.65 seconds
```

How to read it: the result is the same six open ports as a plain SYN scan. The source-port trick does not change what the scanner finds; it changes whether the firewall lets the

probes through at all, which matters when the firewall normally blocks traffic from the scanner's usual source port range.

Real-world use: A tester's SYN scan against a segment is fully blocked (every port filtered), because the firewall drops traffic from high, non-standard source ports. Re-running the same scan with -g 53 (pretending to be DNS traffic) gets through, revealing six open ports that were otherwise hidden.

Tools and commands used here: Nmap (-g, --source-port), Hping3 (-s or --baseport for a fixed source port, ADDED CONTEXT), Colasoft Packet Builder and NetScanTools Pro (manual crafting, see 3.12).

AI angle (ADDED CONTEXT): Ask an LLM: "My scan of 10.10.1.0/24 is fully blocked. The firewall likely trusts DNS and HTTP traffic. Give me Nmap commands that try spoofing the source port to 53 and to 80, and explain the -g flag." A good answer names -g (or --source-port) and warns that this only works if the firewall's rule genuinely keys off the source port rather than deeper packet inspection.

Countermeasures and defense: Configure firewall and router rules so that trust is never based on the source port alone (see port scanning countermeasure 3 in 3.19); use stateful inspection that tracks real connections rather than simple port-based allow rules; keep firmware updated.

EXAM FOCUS:

- How it is asked: "An attacker scans using source port 53 to bypass a firewall rule. What is this technique called?" -> source port manipulation.
- Keywords: "firewall configured to allow packets from well-known ports", "-g or --source-port" -> source port manipulation.
- Trap: this technique manipulates the scanner's own source port, not the destination port being probed. Do not confuse it with simply scanning port 80 as a target.
- Look-alike: source port manipulation (fakes the sender's port number) vs IP address spoofing (3.11, fakes the sender's IP address) vs MAC address spoofing (3.17, fakes the sender's hardware address). All three forge a different field of the same packet.

Memory trick: "-g for Gatecrash: walk in wearing the guard's own uniform (port 80, 53, or 21)."

3.11 IP Address Spoofing

What it is (simple words): IP spoofing refers to changing the source IP address of outgoing packets so that the attack appears to be coming from someone else. When the victim replies, the reply goes back to the spoofed address, not to the attacker's real address. Attackers modify the address information and the source address bits in the IP

packet header in order to bypass an IDS or firewall that trusts, blocks, or logs by source IP.

- **Spoofed address:** a fake source IP address placed in the packet header, not the sender's real address.
- **Reflected reply:** any answer the victim sends goes to the spoofed address instead of the real sender, so the real attacker never sees it directly (unless the spoofed address belongs to him or to a victim he is attacking with the reply, as in a reflection attack, Module 10).

Analogy: Sending a nasty letter and writing a neighbor's return address on the envelope. If the recipient replies, the reply lands on the neighbor's doorstep, not the real sender's.

How it works (numbered steps):

1. The attacker picks a fake source IP address to put in his outgoing packets.
2. He crafts and sends packets (often with a tool such as Hping3) using that fake address as the source.
3. The target (victim) believes the packets came from the fake address and may log, block, or reply to that address instead of the real attacker.
4. Any reply from the victim goes to the spoofed address, so the attacker normally cannot see the reply in a basic spoofing attack. This is why spoofing is mostly used for one-way attacks (flooding, evading IP-based access rules, reflection) rather than for scans that need to read back a result.

Diagram on the slide (illustrative world map with arrows):

```
   Attacker                                  Victim IP
  (sends packet,                             address
   spoofed source                            5.5.5.5
   = 7.7.7.7)   ------------------------------>
                                  Reply appears to come
                                  from / go to 7.7.7.7
```

[UNCLEAR on page 5]: the diagram labels both the spoofed address box and a "Real address" box as 7.7.7.7, which is inconsistent as drawn; the most consistent reading is that 7.7.7.7 is the spoofed address the attacker inserts into the packet, the victim's own address is 5.5.5.5, and the "real address" label marks where the spoofed identity 7.7.7.7 actually belongs on the map (i.e., to a third party, not to the attacker). Treat the key fact as: the attacker's packets claim to come from 7.7.7.7, and the victim is 5.5.5.5; any reply from the victim is sent to 7.7.7.7, not to the attacker.

Command (Hping3 example from the slide):

```
hping3 www.certifiedhacker.com -a 7.7.7.7
```

- hping3: the packet-crafting and scanning tool.

- www.certifiedhacker.com: the target host.
- -a 7.7.7.7 (or --spoof 7.7.7.7): set the source IP address field to 7.7.7.7 instead of the attacker's real IP.

ADDED CONTEXT: a fuller Hping3 spoofing example and reading it.

```
sudo hping3 -c 5 -S -p 80 -a 7.7.7.7 www.certifiedhacker.com
```

- -c 5: send 5 packets.
- -S: set the SYN flag (acts like a simple SYN probe).
- -p 80: target port 80.
- -a 7.7.7.7: spoof the source address as 7.7.7.7.

```
HPING www.certifiedhacker.com (eth0 162.241.216.11): S set,
40 headers + 0 data bytes
len=46 ip=162.241.216.11 ttl=55 DF id=0 sport=80 flags=SA
seq=0 win=8192 rtt=38.5 ms
```

Because the source address was spoofed to 7.7.7.7, the real reply (flags=SA, meaning SYN+ACK) is actually sent by the target to 7.7.7.7, not back to the attacker's own machine; the attacker normally would not see this line unless running the capture from the spoofed host itself or sniffing the path. In practice, most internet service providers filter obviously spoofed packets before they ever leave the attacker's network (see egress filtering in 3.20), so this technique mostly works on poorly secured or internal networks today.

> **REMEMBER:** You will not be able to complete the three-way handshake and open a successful TCP connection with spoofed IP addresses. The victim's SYN/ACK goes to the spoofed address, which never sends the final ACK back to the real attacker (unless the attacker also controls or monitors the spoofed address). This is why IP spoofing is used for denial-of-service, reflection, and IDS or firewall evasion, not for scans that need a two-way answer.

Real-world use: An attacker wants to flood a victim without revealing his own IP. He sends SYN packets with thousands of random spoofed source addresses. The victim's half-open connections pile up (a SYN flood, Module 10), and any logs or IDS alerts point to the fake addresses, not the attacker.

Tools and commands used here: Hping3 (-a, --spoof), Nmap (-S, to set a spoofed source for certain scan types, ADDED CONTEXT), Colasoft Packet Builder and NetScanTools Pro (manual header editing, see 3.12).

AI angle (ADDED CONTEXT): An LLM can explain why a spoofed scan will not return readable results to the attacker, and can write a safe lab-only Hping3 command for a test range you own. Example prompt: "Explain, for an authorized lab test, why I cannot

read scan results back when I spoof the source IP with Hping3's -a option." A correct answer should mention that replies are routed to the spoofed address, not to you.

Countermeasures and defense: See the dedicated IP Spoofing Countermeasures list in 3.20 (encrypt traffic, multiple firewalls, no IP-based authentication, random initial sequence numbers, ingress filtering, egress filtering).

EXAM FOCUS:

- How it is asked: "Why can an attacker not complete a TCP connection using a spoofed source IP address?" -> the SYN/ACK (and all further replies) go to the spoofed address, not to the attacker.
- Keywords: "appears to be coming from someone else", "replies go back to the spoofed address" -> IP address spoofing.
- Trap: IP spoofing changes the IP header's source address field. It is a different field from the source port (3.10) and the MAC address (3.17), even though all three are used together in real evasion attempts.
- Look-alike: IP address spoofing (fake source IP, used for DoS, reflection, evasion) vs IP address decoy (3.16, adds many fake source IPs alongside the real scanning IP, specifically to hide which IP is the real scanner during a scan that still needs to receive replies).

Memory trick: "Spoof the IP, lose the reply: the postman delivers the answer to the name on the envelope, not to the person who actually wrote the letter."

3.12 Creating Custom Packets

What it is (simple words): Attackers create custom TCP packets using various packet crafting tools, such as Colasoft Packet Builder and NetScanTools Pro, to scan a target beyond a firewall. Instead of relying on a scanner's built-in probe types, the attacker builds a packet byte by byte, setting any field (flags, TTL, checksum, source and destination addresses and ports) to exactly the value needed to slip past a specific filter rule.

- **Packet crafting tool:** software that lets a user build a network packet field by field and send it, rather than generating it from a fixed scan template.
- **Decode editor:** a view in a packet crafting tool that shows each protocol layer of a packet (Ethernet, IP, TCP) as an editable tree, so the user can change any single field.

Analogy: A scanner such as Nmap is a vending machine with fixed buttons (SYN scan, UDP scan, and so on). A packet crafting tool is a workbench where the attacker builds a one-off packet from raw parts, choosing exactly which flags, addresses, and checksums to use, when no pre-built button does the job.

How it works (numbered steps):

1. The attacker opens a packet crafting tool and creates a new packet.
2. He builds or edits each layer: Ethernet II (source and destination MAC), the network layer (ARP or IP, with TTL, flags, and addresses), and the transport layer (TCP or UDP, with ports, sequence numbers, and flags).
3. He can use the checksum calculator in the tool to insert a valid (or deliberately invalid, see 3.18) checksum.
4. He sends the single crafted packet, or adds several packets to a list and sends them all, to test exactly how the target or an in-between firewall reacts.

Screenshot on the slide (Colasoft Packet Builder): the interface shows a toolbar (Import, Export, Add, Insert, Delete, Decode Editor, Send, Send All), a Decode Editor pane on the left listing Ethernet II, ARP/Internet Protocol, and transport-layer fields in an expandable tree, a Hex Editor pane at the bottom showing the raw bytes of the packet, and a Packet List pane on the right showing queued packets with a Send and a Send All button. The exact field values filled in on the slide are too small to read with confidence [UNCLEAR on page 6]; the layout described here is the general structure of the tool.

ADDED CONTEXT: other packet crafting tools the exam may name alongside these. Scapy (a Python library and interactive tool for crafting and sending arbitrary packets at any protocol layer), hping3 (also a crafting tool, already used for spoofing in 3.11 and fragmentation in 3.9), and Ostinato (another GUI packet crafting and traffic generator). NetScanTools Pro bundles packet crafting together with many other network utilities (see section 4).

ADDED CONTEXT: a Scapy equivalent, to show what "custom packet" means in code.

```
from scapy.all import IP, TCP, send
pkt = IP(dst="10.10.1.11", ttl=128) / TCP(
    sport=4444, dport=80, flags="S")
send(pkt)
```

This builds one IP packet with a custom TTL (128, to look like Windows) carrying a TCP SYN to port 80 from source port 4444, and sends it. Every field is explicit, which is the point of packet crafting: nothing is left to a scanner's default template.

Real-world use: A tester's standard Nmap SYN scan is dropped by a firewall rule that matches Nmap's default packet signature. Using Colasoft Packet Builder, he crafts a SYN packet with a non-default TTL, window size, and TCP options order, which the same firewall rule does not match, and the probe reaches the target.

Tools and commands used here: Colasoft Packet Builder, NetScanTools Pro, Scapy and hping3 (ADDED CONTEXT).

AI angle (ADDED CONTEXT): An LLM can write a Scapy script from a plain description. Example prompt: "Write a Scapy script that sends a single TCP packet to 10.10.1.11 port 443 with the SYN flag, a TTL of 64, and a custom window size of 1024, for an authorized lab test." Read and understand every line before running it, since a crafted packet bypasses the safety checks of a normal scanner.

Countermeasures and defense: Use a stateful firewall that validates packet structure and flag combinations rather than trusting a specific scanner's signature; keep firewall and IDS rule sets and firmware updated (also in 3.19) so they catch hand-built as well as tool-generated probes.

EXAM FOCUS:

- How it is asked: "Which technique lets an attacker build a packet field by field to bypass a firewall rule that blocks known scanner signatures?" -> creating custom packets with a packet crafting tool.
- Keywords: "packet crafting tools", "Colasoft Packet Builder", "NetScanTools Pro" -> custom packet creation.
- Trap: packet crafting is the general technique; fragmentation (3.9), source port manipulation (3.10), IP spoofing (3.11), and bad checksums (3.18) are all specific tricks that a packet crafting tool can also be used to perform. A question that names a specific trick wants that specific answer, not the general "custom packets" label.
- Look-alike: packet crafting tools (build any packet from scratch) vs Nmap (runs pre-built scan templates, though several accept flags that imitate hand-crafting, such as -f, -g, -D, --badsum).

Memory trick: "Colasoft builds from Scratch; Nmap picks from a Menu."

3.13 OS Discovery / Banner Grabbing

What it is (simple words): Banner grabbing, also called OS fingerprinting in this context, is the method used to determine the operating system running on a remote target system. There are two types of banner grabbing: active and passive. Identifying the OS used on the target host allows an attacker to figure out the vulnerabilities the system possesses and the exploits that might work on it, to carry out further attacks.

- **Banner:** the short text a service sends when a client connects, often naming the product and version, for example "220 Microsoft FTP Service" on an FTP port.
- **Fingerprinting:** identifying a product (here, an OS or service) from the distinctive pattern of its responses, the way a human fingerprint identifies a person.

Analogy: Active banner grabbing is knocking on a door and listening carefully to how it creaks, then matching that creak to a catalog of door models. Passive banner grabbing is simply standing nearby and noting what the mail carrier calls out as he reads the name on each parcel, without knocking on anything yourself.

How it works, active banner grabbing (numbered steps):

1. The attacker sends specially crafted packets to the remote OS.
2. He notes the exact responses: TTL, window size, TCP option order, how the stack reacts to malformed or unusual packets, and more.
3. He compares the responses with a database of known OS fingerprints (for example, Nmap's built-in nmap-os-db).
4. Because different operating systems implement small details of the TCP/IP stack differently, the responses vary enough to tell OS families and sometimes exact versions apart.

Command (ADDED CONTEXT, active OS fingerprinting with Nmap):

```
sudo nmap -O -v 10.10.1.11
sudo nmap -A 10.10.1.11
```

- -O: run active OS detection by sending a series of TCP, UDP, and ICMP probes.
- -A: aggressive mode; runs -O (OS detection), -sV (version detection), -sC (default scripts), and traceroute together.

Sample output (illustrative):

```
Running: Microsoft Windows 10|11|2016|2019|2022
OS CPE: cpe:/o:microsoft:windows_10 ...
OS details: Microsoft Windows 10 1809 - 21H2, ...
Network Distance: 1 hop
```

How to read it: "Running" lists the OS family Nmap believes fits best, sometimes with several close versions. "OS details" refines the guess. "Network Distance" is the hop count Nmap measured, useful for sanity-checking a TTL-based guess from 3.8.

How it works, passive banner grabbing (three techniques from the slide):

1. **Banner grabbing from error messages.** Error messages provide information such as the type of server, type of OS, and SSL tool used by the target remote system. For example, a misconfigured web server's error page might print "Apache/2.4.41 (Ubuntu)" in the footer, or an SSL error might name the exact TLS library in use.
2. **Sniffing the network traffic.** Capturing and analyzing packets from the target (as in 3.8) enables an attacker to determine the OS used by the remote system, without sending the target anything extra.
3. **Banner grabbing from page extensions.** Looking for an extension in the URL may assist in determining the application's version. Example from the slide: .aspx => IIS server and Windows platform. Other common examples (ADDED CONTEXT): .php often implies a Linux or Windows server running PHP (Apache or Nginx common on Linux, IIS possible on Windows); .jsp implies a Java application server such as Tomcat; .cfm implies Adobe ColdFusion.

> **REMEMBER:** The slide notes "We will discuss passive banner grabbing in later modules." Passive banner grabbing from HTTP headers and error pages is covered in more depth when the course reaches web server and web application hacking (Modules 13 and 14).

ADDED CONTEXT: active banner grabbing with everyday tools. Netcat and Telnet can manually grab a banner by opening a raw connection to a port and reading what the service says first.

```
nc -nv 10.10.1.11 21
telnet 10.10.1.11 80
```

```
220 Microsoft FTP Service
```

A plain connection to port 21 that immediately prints "220 Microsoft FTP Service" is itself a simple form of active banner grabbing, even without a dedicated OS-fingerprinting tool.

Real-world use: A tester cannot run -O because a firewall strips the probes Nmap needs for active fingerprinting. He instead grabs the web server's error page, sees "Microsoft-IIS/10.0" in the response headers, and infers a modern Windows Server release without sending a single OS-detection probe.

Tools and commands used here: Nmap (-O, -A), Wireshark and tcpdump (passive sniffing), Netcat and Telnet (manual banner reads).

AI angle (ADDED CONTEXT): Paste a raw HTTP response header or an FTP banner into an LLM and ask: "What operating system and server software does this banner suggest, and what should I check next?" The model can usually name the product and point toward relevant CVEs, but confirm any CVE number in the NVD before relying on it.

Countermeasures and defense: Edit or remove banners and error messages that reveal the product, version, or OS (strip the Server header, use generic error pages, rename default file extensions where practical); disable or filter the probes active fingerprinting tools rely on (where this does not break legitimate diagnostics); keep version information out of publicly visible pages.

EXAM FOCUS:

- How it is asked: "An attacker learns the target's OS by reading an IIS error page, without sending any crafted probes. Which technique is this?" -> passive banner grabbing (from error messages).
- Keywords: "specially crafted packets", "compared with a database" -> active banner grabbing. "Error messages", "sniffing traffic", "page extensions" -> passive banner grabbing.

- Trap: the .aspx -> IIS/Windows example is a specific, memorized fact the exam likes to test directly. Do not confuse it with .php, .jsp, or .cfm.
- Look-alike: active banner grabbing (sends crafted packets, like -O) vs the TTL/window-size method in 3.8 (reads values from an ordinary reply, no special crafted probe needed, technically also passive sniffing in nature) vs passive banner grabbing proper (reads error pages, sniffed traffic, or URL extensions).

Memory trick: "Active KNOCKS (sends crafted packets); Passive LISTENS (reads errors, sniffs traffic, reads the extension)."

3.14 Scanning Beyond IDS and Firewall (Overview)

Note on order: this slide (25) is the menu page that names all ten evasion techniques in the module. In your screenshots it appears on page 7, after several of the individual techniques (packet fragmentation, source port manipulation, IP spoofing, custom packets) had already been shown on pages 4 to 6, and before the rest (source routing, IP decoy, MAC spoofing, randomizing host order, bad checksums) appear on pages 7 to 9. This guide keeps your screenshots' page order, so this overview sits here, but all ten techniques are cross-referenced below regardless of which page introduced them first.

What it is (simple words): Though firewalls and IDSs (Intrusion Detection Systems) can prevent malicious traffic (packets) from entering a network, attackers can still manage to send their intended packets to the target by evading the IDS or firewall, through a set of ten techniques.

- **Firewall:** a device or program that filters traffic in or out of a network according to rules.
- **IDS (Intrusion Detection System):** a device or program that watches traffic and alerts (or, for an IPS, blocks) when it spots a known attack pattern.
- **Evasion:** making malicious or scanning traffic look harmless enough that a firewall or IDS lets it through or fails to flag it.

Analogy: A locked gate with a guard who checks ID cards (firewall and IDS) can still be gotten past by someone who disguises their car, splits their cargo into unmarked boxes, borrows someone else's ID, or sneaks in through a side path the guard never watches. Each evasion technique below is one such disguise.

The ten techniques (icons on the slide), with where each is covered in this guide:

#	Technique	Covered in	One-line idea
1	Packet Fragmentation	3.9	Split the probe into small IP fragments
2	Source Routing	3.15 (below)	Force a specific, non-firewalled path

#	Technique	Covered in	One-line idea
3	Source Port Manipulation	3.10	Fake the sender's port number
4	IP Address Decoy	3.16	Hide the real scanner among fake IPs
5	IP Address Spoofing	3.11	Fake the sender's IP address
6	MAC Address Spoofing	3.17	Fake the sender's hardware address
7	Creating Custom Packets	3.12	Hand-build packets field by field
8	Randomizing Host Order and Sending Bad Checksums	3.18	Scan out of order; send broken checksums
9	Proxy Servers	below (ADDED CONTEXT)	Relay the scan through a middle server
10	Anonymizers	below (ADDED CONTEXT)	Hide the scanner's identity through a web relay

Proxy servers (ADDED CONTEXT, named as an icon on the slide with no further detail slide in the screenshots). A proxy server is a middle system that forwards a connection on behalf of a client. When an attacker routes a scan through one or more proxies, the target sees the proxy's IP address as the source, not the attacker's own address. Nmap supports this with --proxies, taking a comma-separated list of HTTP or SOCKS4 proxy URLs; it affects only NSE scripts and version detection, not the ping, port scanning, or OS detection phases of a scan (verify against your Nmap version). Tools such as ProxyChains can force an entire command's traffic through a chain of proxies at the operating-system level.

```
nmap --proxies http://10.0.0.5:8080 -sV 10.10.1.11
proxychains nmap -sT 10.10.1.11
```

Anonymizers (ADDED CONTEXT, same note as proxy servers above). An anonymizer is a service or tool, often web-based, that strips or hides identifying information (such as the source IP) from a user's traffic, commonly by relaying it through its own servers. Tor (used in Module 02 for dark web footprinting) is the best-known anonymizing network: it routes traffic through several volunteer relays so no single hop knows both who the sender is and what the final destination is. Web-based anonymizer proxies (simple single-hop relay sites) offer a lighter version of the same idea for ordinary browsing.

How the overview works as a whole (numbered steps):

1. The attacker profiles the target's defenses (which ports and protocols the firewall allows, whether the IDS signatures match specific scanner tools).

2. He selects one or more techniques from the ten above that match the gap he has found (for example, a firewall trusting port 80 traffic calls for source port manipulation; a signature-matching IDS calls for packet fragmentation or custom packets).
3. He runs the scan with the chosen evasion option(s) enabled.
4. He compares the evasive scan's results with a plain scan's results (where possible) to measure how much the firewall or IDS was actually blocking.

Real-world use: A tester finds that a plain Nmap SYN scan against a client segment returns every port filtered, while a ping sweep succeeds. He tries, in order, source port manipulation (-g 80), packet fragmentation (-f), and finally IP decoys (-D), and documents which combination, if any, reaches the hosts, as direct evidence for the client's firewall report.

Tools and commands used here: Nmap (most of the ten techniques have a direct Nmap flag), Hping3, Colasoft Packet Builder, NetScanTools Pro, ProxyChains, Tor.

AI angle (ADDED CONTEXT): An LLM can act as a menu: given a description of what the firewall seems to be blocking, it can suggest which of the ten techniques to try first and generate the matching command. Always verify the suggested flag against section 5 before running it, since LLMs sometimes invent flag names that look plausible but do not exist in the real tool.

Countermeasures and defense: See the Port Scanning Countermeasures (3.19) and IP Spoofing Countermeasures (3.20) lists; in general, use stateful, reassembling, signature- and behavior-aware firewalls and IDS/IPS, patch and update them regularly, and do not trust traffic based on a single easily forged field (port number, IP address, or MAC address).

EXAM FOCUS:

- How it is asked: "Which of the following is NOT one of the ten techniques for scanning beyond IDS and firewall listed by EC-Council?" Expect distractors from other modules (such as SQL injection or phishing) mixed with the real ten.
- Keywords: "evade an IDS or firewall", "send intended packets to the target" -> this group of ten techniques as a whole.
- Trap: proxy servers and anonymizers sound alike. A proxy server is usually a specific relay a tool is configured to use for one connection or scan (--proxies). An anonymizer (or an anonymizing network like Tor) usually implies a broader service built specifically to hide identity, often with multiple hops.
- Look-alike: this overview slide groups many distinct techniques; do not answer a specific-technique question with the general label "scanning beyond IDS and firewall" when the question wants the specific one (fragmentation, decoy, and so on).

Memory trick: "Please Send Spies Inside, Masking Custom Routes, Reaching Past Agents" -> Packet fragmentation, Source routing, Source port manipulation, IP decoy, MAC spoofing, Custom packets, Randomize and bad checksums, Proxy servers, Anonymizers (10 items, approximate mnemonic order).

3.15 Source Routing

What it is (simple words): As a packet travels through the nodes (routers) in a network, each router normally examines the destination IP address and chooses the next hop itself to direct the packet toward its destination. Source routing refers to sending a packet to the intended destination with a partially or completely specified route, bypassing firewall- or IDS-configured routers, in order to evade an IDS or firewall. In source routing, the attacker, not the normal routing infrastructure, makes some or all of these forwarding decisions.

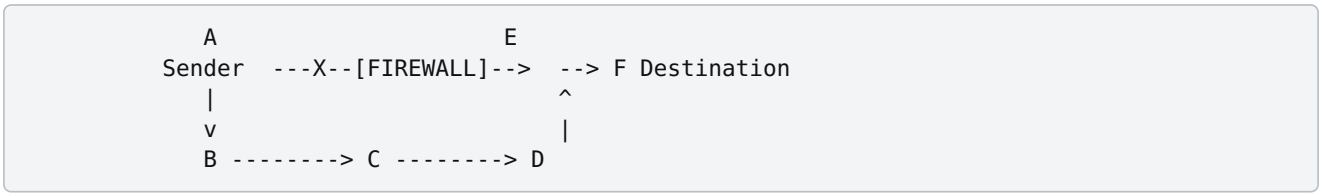
- **Next hop:** the next router a packet is sent to on its way to the final destination.
- **IP option:** an optional field in the IP header that can carry extra instructions, including a source route.
- **Loose source routing:** the sender lists some routers the packet must pass through, but normal routing fills in the rest of the path.
- **Strict source routing:** the sender lists every router the packet must pass through, in exact order, with no other hops allowed.

Analogy: Normally, a courier company decides which depots your parcel passes through. Source routing is like writing on the parcel itself, "you must go through depot B and depot D, skip every other depot", so the parcel avoids a checkpoint the sender knows about at depot C (where the firewall sits).

How it works (numbered steps):

1. The attacker studies the network's layout and finds a firewall- or IDS-protected router on the normal path to the target (shown as the router with the lock icon in the diagram).
2. He crafts a packet that carries IP options specifying some or all of the route it should take, deliberately routing around the protected router.
3. Intermediate routers that honor source routing follow the attacker's specified path instead of their own normal routing table.
4. The packet reaches the destination through a path that never passed through the firewall or IDS that would otherwise have inspected or blocked it.

Diagram on the slide:



As the figure shows, source routing is where the originator (sender A) dictates the eventual route of the traffic. The direct path from A to E is blocked by a firewall-configured router (marked X). Instead, the attacker's packet carries source-route options that send it A -> B -> C -> D -> E -> F, reaching the destination F by a path that never crosses the blocked, firewalled link.

ADDED CONTEXT: Nmap's IP options support.

```
sudo nmap --ip-options "L 10.10.1.1 10.10.1.2" 10.10.1.11
sudo nmap --ip-options "S 10.10.1.1 10.10.1.2" 10.10.1.11
```

- --ip-options: pass IP options directly, including source routing. The letter L requests loose source routing and S requests strict source routing, each followed by a space-separated list of IP addresses that must appear on the path.
- Nmap can also request record-route or record-timestamp options with the shortcut letters R and T.

Real-world use: A penetration tester, with the client's written approval, demonstrates that an edge router still honors source-routed IP options, letting a crafted packet bypass a firewall segment entirely. The finding becomes a priority recommendation, because most production networks should disable source routing at every router they control.

Tools and commands used here: Nmap (--ip-options), Hping3 and Scapy (manual IP option crafting, ADDED CONTEXT), Colasoft Packet Builder and NetScanTools Pro (section 3.12).

AI angle (ADDED CONTEXT): Ask an LLM: "Explain the difference between loose and strict source routing in the IPv4 header, and why most modern routers disable both by default." A good answer should mention that source routing is rarely seen in practice today precisely because it is a well-known evasion and spoofing-support risk, and that many internet routers block the most dangerous IP options outright.

Countermeasures and defense: Disable IP source routing on routers and hosts (most modern operating systems and routers disable it by default, but always verify the configuration); drop packets that carry source-route IP options at the network perimeter; log and alert on any packet that attempts to use these options.

EXAM FOCUS:

- How it is asked: "Which evasion technique lets the sender, rather than the network's own routers, decide the path a packet takes to bypass a firewalled router?" -> source routing.
- Keywords: "originator dictates the route", "partially or completely specified route", "without firewall-/IDS-configured routers" -> source routing.
- Trap: source routing changes the path the packet takes through the network. It does not change any field that identifies the sender (that is IP spoofing, 3.11, and

MAC spoofing, 3.17) or the apparent sending port (that is source port manipulation, 3.10).
- Look-alike: source routing (controls the path) vs IP address decoy (3.16, controls which IP looks like it is scanning, not the path) vs proxy servers (3.14, relays through one specific extra hop chosen by the tool, not a full attacker-specified router-by-router path encoded in IP options).

Memory trick: "Source Routing: the Sender draws the map (A to B to C to D to E), skipping the guarded gate."

3.16 IP Address Decoy

What it is (simple words): The IP address decoy technique refers to generating, or manually specifying, the IP addresses of decoys in order to evade an IDS or firewall. It appears to the target that the decoys, as well as the real host, are all scanning the network at the same time. This makes it difficult for the IDS or firewall to determine which IP address was actually doing the scanning and which addresses were just decoys.

- **Decoy:** a fake source IP address added to a scan alongside (or hiding) the attacker's real address, so the target sees many apparent scanners instead of one.

Analogy: Instead of one person sneaking up to a door and testing the handle, a crowd of people (some real accomplices, some just mannequins wheeled into place) all approach different doors in the street at once. The homeowner's camera records twenty people; picking out which one actually touched the lock is far harder.

How it works (numbered steps):

1. The attacker chooses to run a scan with decoys, either generating them randomly or listing specific addresses.
2. Nmap sends the same probes from the real scanning host and, in parallel, crafts packets that appear to come from each decoy address.
3. The target (and any IDS or firewall watching it) sees packets arriving from several different source IPs, all probing the same ports.
4. Because the real scanner's packets are mixed in with the decoys' packets, picking the real attacker out of the group requires more than just looking at the list of source IPs; the defender needs a method such as checking which IPs are even routable or reachable from the attacker's side.

Decoy scanning using Nmap, two options from the slide:

```
sudo nmap -D RND:10 10.10.1.11
sudo nmap -D decoy1,decoy2,decoy3,ME,decoy4 10.10.1.11
```

- nmap -D RND:10 [target]: generates a random number of decoys (here, 10) automatically.

- nmap -D decoy1,decoy2,decoy3,..etc [target]: manually specifies the exact IP addresses of the decoys to use.
- Optionally, the keyword ME can be placed in the manual decoy list to control exactly where the real scanning address appears in the sequence (ADDED CONTEXT; if ME is omitted, Nmap places the real address at a random position in the list).

Sample output (as seen in the module's screenshot, shortened):

```
nmap -D RND:10 10.10.1.11 - Parrot Terminal
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-03-08
Nmap scan report for 10.10.1.11
Host is up (0.00067s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT     STATE SERVICE
80/tcp   open  http
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
3389/tcp open  ms-wbt-server
MAC Address: 00:15:5D:01:00:00 (Microsoft)
Nmap done: 1 IP address (1 host up) scanned in 4.93 seconds
```

How to read it: the command (-D RND:10) and the exact timestamp are small in the capture [UNCLEAR on page 8]; the output shown is the best reading. Note that the decoy flag does not change what the scanner itself discovers (five open ports here); it changes how many apparent source IP addresses the target's logs will show for the same set of probes. A defender reviewing logs on 10.10.1.11 would see traffic that looks like it came from 10 (or more, including the real one) different IP addresses, all probing the same ports at nearly the same time.

> **TRAP:** Decoys must be plausible, reachable hosts, or they stand out as fake. An address that clearly could not route from the attacker's own network position is a giveaway. The decoys' own "replies" are never seen by the attacker; he still learns the result only from his own real packets mixed into the crowd.

Real-world use: During an authorized assessment, a tester runs a decoy scan so that the client's own detection logs will show ten apparent source IPs scanning simultaneously. The exercise is used afterward to test whether the client's SOC (Security Operations Center) can correctly flag the real attacker's IP out of the noise, as part of a detection capability test.

Tools and commands used here: Nmap (-D RND:N, -D with a manual list, optionally with ME).

AI angle (ADDED CONTEXT): Ask an LLM: "Explain how Nmap's decoy scanning option works and why it mainly helps hide the scanner's identity rather than hide that a scan is happening at all." A correct answer notes that decoys add noise around the real

scanner's IP, but an alert defender who notices many simultaneous scanners from implausible or unrelated networks may still suspect decoy-based evasion.

Countermeasures and defense: Correlate scan alerts with routing plausibility (which source IPs could actually reach you from where they claim to be); use anti-spoofing and anti-scanning firewall rules (see Port Scanning Countermeasure 8 in 3.19); rely on more than source IP alone when attributing a scan, such as timing correlation and traffic volume per source.

EXAM FOCUS:

- How it is asked: "Which Nmap option generates a random number of decoy source addresses for a scan?" -> nmap -D RND:N.
- Keywords: "appears that decoys as well as the host(s) are scanning", "difficult for the IDS or firewall to determine which IP was actually scanning" -> IP address decoy.
- Trap: -D RND:10 and -D decoy1,decoy2,... are the two forms shown on the slide; do not confuse -D (decoys) with -g (source port manipulation, 3.10) or -a / --spoof (single spoofed source, 3.11, which replaces the real address rather than hiding it among many).
- Look-alike: IP address decoy (adds many fake source IPs so the real scanner blends in, used during active scans that still need results) vs IP address spoofing (3.11, replaces the real source IP entirely with one fake address, usually sacrificing the ability to see the reply).

Memory trick: "Decoy = Disguise by Crowd. One real scanner hides inside a crowd of fake ones."

3.17 MAC Address Spoofing

Note: the screenshot for this technique shows only the command, its output, and a warning banner; no slide title or bullet text is visible in the capture [UNCLEAR on page 8]. The topic and definition below are reconstructed from the command itself (--spoof-mac) and from the "MAC Address Spoofing" icon already named on the overview slide (3.14, item 6).

What it is (simple words): MAC address spoofing means changing the hardware (MAC) address that a scanning tool uses for its outgoing frames, so that the frames appear to come from a different network card than the one actually running the scan. This can help an attacker evade access controls or logging that key off a device's MAC address, or impersonate a trusted vendor's hardware on a local network.

- **MAC address:** a 48-bit hardware address burned into (or configurable on) a network interface, usually written as six pairs of hex digits, for example 00:00:97:A2:AE:71.

- **OUI (Organizationally Unique Identifier):** the first three bytes of a MAC address, assigned to a specific manufacturer; Nmap can look up a vendor name (such as "Dell") and fill in a matching OUI automatically.

How it works (numbered steps):

1. The attacker chooses a MAC address, a MAC prefix, or a vendor name to impersonate.
2. Nmap (or another crafting tool) builds its raw Ethernet frames using that address instead of the real network card's address.
3. Any local-network device that logs or filters by MAC address (such as a switch with port security, or a network access control system) sees the spoofed vendor's address instead of the real one.
4. Because using a spoofed MAC requires sending raw Ethernet frames, this only works for scan types that build their own raw packets; it does not work for a plain TCP connect scan, which relies on the operating system's normal networking stack.

Command from the slide:

```
sudo nmap -sT -Pn --spoof-mac Dell 10.10.1.11
```

- -sT: TCP connect (full open) scan.
- -Pn: skip host discovery, treat the target as up.
- --spoof-mac Dell: spoof the MAC address using Dell's OUI, with the rest of the address filled in randomly. A value of 0 asks Nmap to pick a completely random MAC; a full or partial hex string can also be given directly.

Sample output (as seen on the slide, shortened):

```
nmap -sT -Pn --spoof-mac Dell 10.10.1.11 - Parrot Terminal
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-03-12
Spoofing MAC address 00:00:97:A2:AE:71 (Dell EMC)
You have specified some options that require raw socket
access. These options will not be honored for TCP Connect
scan.
Nmap scan report for 10.10.1.11
Host is up (0.00030s latency).
Not shown: 994 closed tcp ports (conn-refused)
PORT     STATE SERVICE
21/tcp   open  ftp
80/tcp   open  http
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
3389/tcp open  ms-wbt-server
Nmap done: 1 IP address (1 host up) scanned in 1.74 seconds
```

How to read it line by line:

- "Spoofing MAC address 00:00:97:A2:AE:71 (Dell EMC)": Nmap confirms the chosen fake MAC address and the vendor name it matched from the OUI database.

- The warning "these options will not be honored for TCP Connect scan" is the key lesson of this screenshot: because -sT (connect scan) uses the operating system's normal socket interface rather than Nmap's own raw packet engine, Nmap cannot actually apply the spoofed MAC address to this particular scan type. The warning is Nmap telling the user that the --spoof-mac request was effectively ignored here.
- "Not shown: 994 closed tcp ports (conn-refused)": the "conn-refused" reason, instead of "reset", is typical wording for a connect scan, matching -sT as used here.
- The result (six open ports) is the same set seen in earlier screenshots of this target, confirming that the scan itself still ran normally, just without the MAC spoofing actually taking effect.

> **TRAP:** This screenshot is as much a lesson in a limitation as in a technique: --spoof-mac (like several raw-packet-dependent options) needs a scan type that uses Nmap's own raw socket engine, such as -sS, -sF, -sX, or -sN, not -sT. Pairing --spoof-mac with -sT, as shown here, produces a clear warning and the spoofing silently does not happen.

Real-world use: A tester on a local segment wants to see whether a switch's port-security or NAC (Network Access Control) rules trust a specific vendor's devices (for example, approved Dell laptops). He runs a raw-socket scan type (such as -sS) with --spoof-mac Dell, confirms the switch allows the traffic through under the spoofed vendor identity, and reports the gap in the NAC policy.

Tools and commands used here: Nmap (--spoof-mac, requires -sS, -sU, or another raw-packet scan type rather than -sT), macchanger (a dedicated Linux tool for changing a network interface's MAC address at the OS level, ADDED CONTEXT).

AI angle (ADDED CONTEXT): Ask an LLM: "Why does Nmap's --spoof-mac option not work with a TCP connect scan (-sT), and which scan types would let it actually spoof the MAC address?" A good answer should explain the raw-socket requirement and suggest -sS or a similar raw scan type instead.

Countermeasures and defense: Use 802.1X port-based network access control instead of trusting MAC addresses alone (MAC addresses are trivially spoofable); enable switch port security features that limit or alert on unexpected MAC changes; do not rely on MAC address allow-listing as a sole network access control.

EXAM FOCUS:

- How it is asked: "An attacker runs nmap -sT --spoof-mac Dell against a target and sees a warning that the MAC spoofing was not honored. Why?" -> TCP connect scan (-sT) uses the OS's own sockets, not Nmap's raw packet engine, so raw-packet-only options like --spoof-mac have no effect with -sT.
- Keywords: "--spoof-mac", "OUI", "vendor name" -> MAC address spoofing.

- Trap: do not assume --spoof-mac always works just because it is specified; the scan type chosen alongside it matters a great deal, as this screenshot demonstrates directly.
- Look-alike: MAC address spoofing (fakes the hardware address, relevant mainly on the local link / same broadcast domain) vs IP address spoofing (3.11, fakes the network-layer address, relevant across routed networks).

Memory trick: "MAC spoofing needs RAW power (-sS, -sU, and friends); -sT (plain Connect) cannot carry it."

3.18 Randomizing Host Order and Sending Bad Checksums

What it is (simple words): This slide covers two separate evasion tricks side by side.

- **Randomizing host order:** attackers scan the number of hosts in the target network in random order, rather than scanning them sequentially (such as .1, .2, .3, and so on), to scan an intended target that is behind a firewall without producing an obvious sequential pattern.
- **Sending bad checksums:** attackers send packets with bad, or bogus, TCP/UDP checksums to the intended target, in order to avoid certain firewall rulesets.

Analogy: Randomizing host order is like a thief casing a street by knocking on houses in no particular order (14, then 3, then 29) instead of walking straight down the row from 1 to 30, so a watchful neighbor cannot predict which house comes next. Sending bad checksums is like mailing a parcel with a deliberately smudged, unreadable return-address stamp: a lazy guard who only checks that the stamp "looks present" lets it through, while a careful guard who actually verifies the stamp would reject it as damaged, revealing which guard is paying attention.

How randomizing host order works (numbered steps):

1. The attacker has a range or list of target hosts to scan.
2. Instead of scanning them in their natural numeric or listed order, the scanner shuffles the order of up to a set group size before scanning.
3. Monitoring systems that look for a steady, sequential sweep pattern (.1, .2, .3, ...) are less likely to flag the random-order traffic as an obvious scan.

```
sudo nmap --randomize-hosts 10.10.1.0/24
```

- --randomize-hosts: tells Nmap to shuffle each group of up to 16,384 hosts before scanning them (ADDED CONTEXT; the exact group size can be changed at compile time, but 16,384 is the documented default, verify for your version).

Sample output (as seen on the slide, shortened, single-host screenshot shown for illustration of the flag; the technique matters most across a range):

```
nmap --randomize-hosts 10.10.1.11 - Zenmap
Starting Nmap 7.94 ( https://nmap.org ) at 2024-03-12
Nmap scan report for 10.10.1.11
Host is up (0.00061s latency).
Not shown: 994 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
80/tcp   open  http
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
3389/tcp open  ms-wbt-server
MAC Address: 00:15:5D:01:00:00 (Microsoft)
Nmap done: 1 IP address (1 host up) scanned in 1.63 seconds
```

How to read it: against a single host the result is identical to a plain scan (six open ports in 1.63 seconds), because there is only one host to "randomize"; the real effect only shows up when scanning a range of many hosts, where the order in which each one is probed becomes unpredictable.

How sending bad checksums works (numbered steps):

1. The attacker deliberately sets an invalid TCP or UDP checksum on each outgoing packet.
2. A target's own real TCP/IP stack checks the checksum, finds it wrong, and silently drops the packet, so a genuine host will never actually respond to a bad-checksum probe.
3. However, some simple firewalls and packet filters only check whether a packet matches a rule (ports, flags, addresses) and do not bother verifying the checksum themselves before passing or logging it.
4. If any reply does come back after a bad-checksum probe, it is almost certainly from a firewall or an IDS that generated the reply itself, rather than from the real target host, which would have dropped the broken packet. This can reveal the presence and behavior of an in-path security device.

```
sudo nmap --badsum 10.10.1.11
```

- --badsum: asks Nmap to use an invalid TCP, UDP, or SCTP checksum for packets sent to the target.

Sample output (as seen on the slide, shortened):

```
nmap --badsum 10.10.1.11 - Zenmap
Starting Nmap 7.94 ( https://nmap.org ) at 2024-03-12
Nmap scan report for 10.10.1.11
Host is up (0.0004s latency).
All 1000 scanned ports on 10.10.1.11 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
Nmap done: 1 IP address (1 host up) scanned in 23.12 seconds
```

How to read it: every one of the 1000 scanned ports comes back "filtered (no-response)", because the real host's TCP/IP stack correctly drops every packet with an invalid checksum, exactly as RFC-compliant systems should. The scan also takes much longer (23.12 seconds versus 1.63 to 1.65 seconds for earlier scans of the same host), because Nmap must wait out the retransmission timeouts for every port that never answers. If, instead, some ports in a real test did show a reply, that reply is a strong signal that a firewall or IDS is itself answering on the target's behalf without verifying the checksum, which is valuable information about the security device sitting in front of the host.

Real-world use: During a firewall assessment, a tester deliberately sends --badsum probes at a server that is known (from earlier scans) to have several open ports. Because every probe now shows "filtered (no-response)" instead of "open", she confirms that the server's own OS correctly validates checksums; she then repeats the same test against the border firewall's management interface and finds that it answers anyway, revealing that the firewall's own stack does not verify checksums before responding, a useful finding for the report.

Tools and commands used here: Nmap (--randomize-hosts, --badsum).

AI angle (ADDED CONTEXT): Ask an LLM: "Explain what Nmap's --badsum option tests for, and why a reply to a bad-checksum probe is meaningful even though the probe is technically broken." A good answer notes that a correctly implemented TCP/IP stack must drop the packet, so any reply points to a firewall or IDS device that is not checking checksums itself.

Countermeasures and defense: See Port Scanning Countermeasures (3.19), especially items 1, 2, 3, and 6 to 8, which call for properly configured firewall and IDS rules, routing and filtering that cannot be bypassed, and regular checks of your own address space using the same TCP, UDP, and ICMP probes attackers would use.

EXAM FOCUS:

- How it is asked: "Which Nmap option sends packets with deliberately invalid TCP or UDP checksums to test firewall rule handling?" -> --badsum. "Which option shuffles the order in which a range of hosts is scanned?" -> --randomize-hosts.
- Keywords: "bogus checksums", "avoid certain firewall rulesets" -> bad checksums. "Random order", "scan the number of hosts... in random order" -> randomizing host order.
- Trap: a bad-checksum probe never gets a real reply from the actual target host; any reply indicates an intermediate device (firewall or IDS), not the target itself.
- Look-alike: sending bad checksums (tests whether a device validates checksums before responding) vs the ACK or window scan (3.6, reads legitimate RST replies to learn about firewall filtering) -- both probe firewall behavior, but bad checksums rely on an invalid packet being mishandled, while ACK/window scans use entirely valid packets.

Memory trick: "Randomize the Order so no one sees a straight line; Break the Checksum so only a careless guard answers."

3.19 Port Scanning Countermeasures

What it is: Eight defensive measures an organization should take against port scanning, listed on the slide.

1. Configure firewall and IDS rules to detect and block probes.
2. Run port scanning tools against hosts on the network to determine whether the firewall properly detects port scanning activity.
3. Ensure that the mechanisms used for routing and filtering at the routers and firewalls, respectively, cannot be bypassed using a particular source port or source routing methods.
4. Ensure that the router, IDS, and firewall firmware are updated to their latest releases and versions.
5. Use a custom rule set to lock down the network and block unwanted ports at the firewall.
6. Filter all ICMP messages (that is, inbound ICMP message types and outbound ICMP type 3 unreachable messages) at the firewalls and routers.
7. Perform TCP and UDP scanning, along with ICMP probes, against your organization's IP address space to check the network configuration and its available ports.
8. Ensure that anti-scanning and anti-spoofing rules are properly configured.

Key terms:

- **Rule set:** the ordered list of allow and deny rules a firewall applies to traffic.
- **ICMP type 3:** "destination unreachable"; code 3 specifically means "port unreachable", which attackers use to learn that a UDP port is closed (section 3.6).
- **Anti-scanning rule:** a firewall or IDS rule that specifically detects scan-like traffic patterns (many ports touched quickly from one source, unusual flag combinations, and so on) and blocks or alerts on them.
- **Anti-spoofing rule:** a rule that drops packets whose source address could not plausibly have come from the interface it arrived on (closely related to ingress and egress filtering in 3.20).

How the countermeasures map to the techniques earlier in this module:

Countermeasure	Stops which technique(s)
1 Firewall/IDS rules to detect and block probes	All scan types (3.6)
2 Test own network with scanning tools	Confirms that 1 actually works
3 Routing/filtering not bypassable by source port or route	Source port manip. (3.10), source routing (3.15)
4 Update router, IDS, firewall firmware	Known evasion bugs in older firmware
5 Custom rule set, block unwanted ports	Open ports that should not be exposed

Countermeasure	Stops which technique(s)
6 Filter ICMP inbound and outbound type 3	ICMP host discovery (3.5), UDP scan closed-port signal (3.6)
7 Scan your own IP space with TCP/UDP/ICMP	Confirms the real exposed footprint
8 Anti-scanning and anti-spoofing rules	Decoys (3.16), spoofing (3.11, 3.17), bad checksums (3.18)

Real-world use: After a penetration test shows that source port manipulation (-g 80) bypassed the client's firewall, the client's network team tightens its rule set so that trust is based on full connection state rather than the source port alone (countermeasure 3), then re-tests with the same technique to confirm the fix (countermeasure 2).

Tools and commands used here: The same tools used to attack (Nmap, Hping3) are the right tools to test your own defenses with, under countermeasures 2 and 7.

AI angle (ADDED CONTEXT): Ask an LLM to turn this eight-item list into a firewall change-review checklist for a specific vendor's device (for example, a specific firewall platform), including the exact configuration commands. Always test any suggested configuration in a lab before applying it to a production firewall.

Countermeasures and defense: This section is itself the countermeasure list for the module's scanning and evasion techniques (3.5 through 3.18).

EXAM FOCUS:

- How it is asked: "Which of the following is NOT a recommended port scanning countermeasure?" Expect one plausible-sounding distractor (for example, "disable the firewall during business hours") mixed among the eight real items.
- Keywords: "filter ICMP type 3", "anti-scanning and anti-spoofing rules", "scan your own IP space" -> this list.
- Trap: countermeasure 6 specifically calls out ICMP type 3 (destination unreachable), not all ICMP types; do not pick an answer that says "block all ICMP everywhere" as if that were the exact recommendation.

Memory trick: "Firewalls Test Routes, Update, Lock, Filter, Probe, and Guard" -> Firewall/IDS rules, Test with scanners, Routing/filtering not bypassable, Update firmware, Lock down with custom rules, Filter ICMP, Probe your own space, Guard with anti-scan/ anti-spoof rules.

3.20 IP Spoofing Countermeasures

What it is: Six defensive measures against IP address spoofing, listed on the slide.

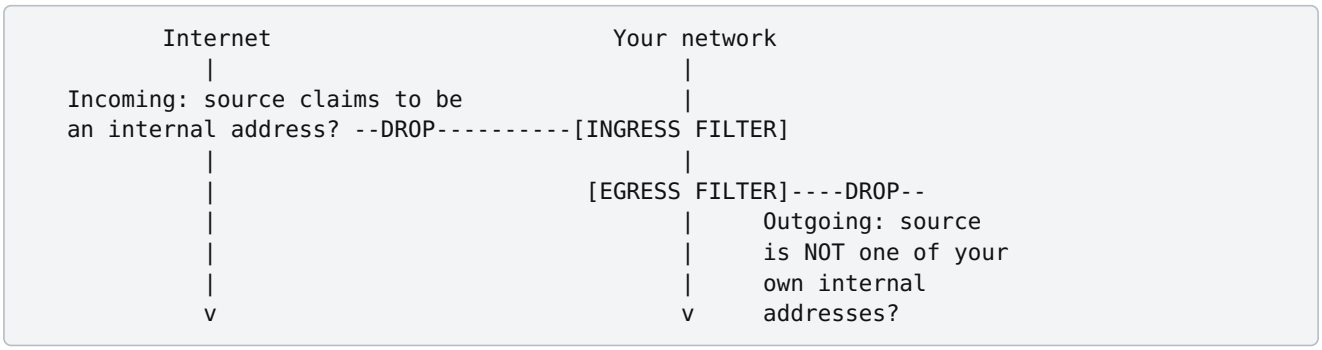
1. Encrypt all the network traffic using cryptographic network protocols such as IPsec, TLS, SSH, and HTTPS.
2. Use multiple firewalls to provide a multi-layered depth of protection.
3. Do not rely on IP-based authentication.

4. Use a random initial sequence number to prevent IP spoofing attacks based on sequence number spoofing.
5. Ingress filtering: use routers and firewalls at your network perimeter to filter incoming packets that appear to come from an internal IP address.
6. Egress filtering: filter all outgoing packets with an invalid local IP address as the source address.

Key terms:

- **IPsec, TLS, SSH, HTTPS:** cryptographic protocols that authenticate and often encrypt traffic, so even if a packet's source IP is spoofed, the attacker cannot forge the cryptographic proof needed to be accepted as a legitimate party.
- **IP-based authentication:** trusting a connection simply because it comes from an allowed IP address (for example, an old-style .rhosts or firewall allow-list by source IP), with no password or cryptographic check.
- **Initial sequence number (ISN):** the starting sequence number a TCP stack picks when opening a new connection. If an attacker can predict a victim's next ISN, he can complete a spoofed TCP handshake blindly, without ever seeing the real replies; a properly randomized ISN defeats this.
- **Ingress filtering:** checking packets as they enter your network from the outside.
- **Egress filtering:** checking packets as they leave your network on their way out.

How it works, ingress and egress filtering together:



1. Ingress filtering sits at the point where traffic enters your network from outside. If an incoming packet claims a source address that belongs to your own internal address space, it cannot be genuine (it would mean an outside packet is pretending to be one of your own internal hosts), so it is dropped.
2. Egress filtering sits at the point where traffic leaves your network. If an outgoing packet's source address is not actually one of your own assigned internal addresses, it cannot be genuine traffic from inside your network, so it is dropped before it ever reaches the internet. This is one of the most effective ways to stop your own network from being used to launch spoofed attacks against someone else.

Real-world use: An ISP implements egress filtering (also known more broadly as BCP 38 / source address validation, ADDED CONTEXT) on its customer-facing links, so that

any customer's outgoing packet with a forged source address is dropped at the ISP's own edge, before it can ever reach the wider internet to attack a third party. This single control removes a large share of the internet's capacity for spoofed denial-of-service traffic.

Tools and commands used here: Router and firewall access-control lists (ACLs) implement ingress and egress filtering; IPsec, TLS/SSL, SSH, and HTTPS implement the encryption and authentication countermeasure.

AI angle (ADDED CONTEXT): Ask an LLM: "Write an example Cisco IOS access list that implements egress filtering, dropping any outgoing packet whose source address is not within our own 203.0.113.0/24 range." Review and test any generated ACL carefully in a lab before deploying it, since an incorrect filtering rule can take down legitimate traffic.

Countermeasures and defense: This section is itself the countermeasure list for IP address spoofing (3.11).

EXAM FOCUS:

- How it is asked: "Which filtering technique drops outbound packets whose source address does not belong to your own internal address range?" -> egress filtering. "Which filtering technique drops inbound packets that falsely claim to be from an internal address?" -> ingress filtering.
- Keywords: "random initial sequence number" -> defends against blind spoofing of the TCP handshake. "Do not rely on IP-based authentication" -> removes the whole incentive for an attacker to spoof an address in the first place.
- Trap: ingress filtering protects your own network from spoofed traffic coming in; egress filtering protects the rest of the internet from spoofed traffic going out of your network. They are often tested as a pair, and it is easy to swap which direction is which.
- Look-alike: ingress filtering vs egress filtering (direction of travel is the only difference) vs the Port Scanning Countermeasures' anti-spoofing rules (3.19, item 8, a broader, less specific version of the same idea applied to scanning defenses generally).

Memory trick: "IN-gress checks what comes IN claiming to be you; E-gress checks what goes Elsewhere claiming to be you (but is not)."

4. Tool Encyclopedia

All tools below are used in the scanning phase (phase 2) unless stated otherwise. Tools not named in the screenshots are marked (added).

4.1 Core Scanners and Packet Crafters

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
Nmap	Host discovery, port/service/OS scanning, scripting	CLI, cross-platform	Scanning	Almost every flag question
Zenmap	Official GUI front end for Nmap	GUI, cross-platform	Scanning	"In Zenmap, -sV detects..."
Hping3	Crafts and sends custom TCP/UDP/ICMP packets	CLI, Linux	Scanning	IP spoofing, -a, --scan
Metasploit	Exploitation framework with scanner modules	CLI/console, cross-platform	Scanning, exploitation	"auxiliary/scanner" modules
NetScanTools Pro	All-in-one network scanning and crafting suite	Windows GUI	Scanning	"Creating Custom Packets"
Colasoft Packet Builder	Builds and sends custom packets field by field	Windows GUI	Scanning	"packet crafting tool"
Scapy (added)	Python library to craft/send any packet	CLI/library, cross-platform	Scanning	"craft packets in code"
Ostinato (added)	GUI packet crafting and traffic generator	GUI, cross-platform	Scanning	Packet crafting alternative
ProxyChains (added)	Forces a tool's traffic through proxy chains	CLI, Linux	Evasion	"relay through proxies"

4.2 Host and Ping Sweep Tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
Angry IP Scanner	Fast cross-platform IP and port scanner	GUI, cross-platform	Host discovery	"ping sweep tool"
SolarWinds Engineer's Toolset	Suite including ping sweep and network tools	Windows GUI	Host discovery	"ping sweep tool"

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
NetScanTools Pro	Also used for ping sweeps and discovery	Windows GUI	Host discovery	"ping sweep tool"
Colasoft Ping Tool	Simple graphical ping sweep tool	Windows GUI	Host discovery	"ping sweep tool"
Advanced IP Scanner	Free LAN scanner, shows live hosts and MACs	Windows GUI	Host discovery	"ping sweep tool"
OpUtils	SolarWinds network monitoring and scanning suite	Web/Windows	Host discovery	"ping sweep tool"

4.3 OS Discovery, Banner Grabbing, and Packet Analysis

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
Wireshark	Captures and inspects packets, reads TTL/window	GUI, cross-platform	OS discovery	"sniff/capture the response"
tcpdump (added)	Command-line packet capture	CLI, Linux/Unix	OS discovery	Flag-letter capture output
Nmap -O / -A (added)	Active OS fingerprinting, aggressive scan	CLI	OS discovery	"-O", "-A"
Netcat (added)	Reads raw banners from any TCP/UDP port	CLI, cross-platform	Banner grabbing	"manually grab a banner"
Telnet (added)	Opens a raw connection to read a service banner	CLI, cross-platform	Banner grabbing	"manually grab a banner"

4.4 Evasion and Anonymity Aids

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
Tor (added)	Anonymizing network, multi-hop relay	Overlay network	Evasion	"anonymizer", multi-hop
macchanger (added)	Changes a NIC's MAC address at the OS level	CLI, Linux	Evasion	"change MAC address"

EXAM TIP: When two tools look right, match the exact verb from the question: "craft a custom packet" -> Colasoft Packet Builder or NetScanTools Pro; "ping sweep" -> Angry IP Scanner or the other five named sweep tools; "sniff TTL and window size" -> Wireshark; "spoof a source IP in a one-off packet" -> Hping3; "run every scan type with one flag-based tool" -> Nmap.

5. Command Sheet

How to read any command below: the first word is the program; words starting with - or -- are options (flags); a flag may take a value right after it; the rest are targets. Most raw-packet Nmap scans need root or administrator rights (use sudo on Linux). Flags marked "verify" depend on your tool version.

5.1 Nmap Host Discovery Flags

Purpose: Find which hosts in a range are alive, without necessarily scanning their ports.

```
nmap -sn [-PE|-PP|-PM|-PR|-PU|-PS[ports]|-PA[ports]|-PO[protos]] TARGET
```

Flag	Meaning	When to use	Example
-sn	Ping scan only, skip port scan	Fast "who is alive" sweep	nmap -sn 10.10.1.0/24
-PE	ICMP echo ping	Simple internal networks	nmap -sn -PE 10.10.1.1
-PP	ICMP timestamp ping	Echo is blocked	nmap -sn -PP 10.10.1.1
-PM	ICMP address mask ping	Echo is blocked	nmap -sn -PM 10.10.1.1
-PR	ARP ping	Local LAN, most reliable	nmap -sn -PR 10.10.1.0/24
-PU[ports]	UDP ping	TCP/ICMP both blocked	nmap -sn -PU53 10.10.1.1
-PS[ports]	TCP SYN ping	ICMP blocked by firewall	nmap -sn -PS80,443 10.10.1.1
-PA[ports]	TCP ACK ping	SYN blocked, ACK allowed	nmap -sn -PA80 10.10.1.1
-PO[protos]	IP protocol ping	Unusual filter rules	nmap -sn -PO1,2,4 10.10.1.1
-Pn	Skip discovery, assume host is up	Discovery itself is blocked	nmap -Pn -sS 10.10.1.1
--disable-arp-ping	Stop automatic ARP on local nets	Force a chosen ping type	--disable-arp-ping

Demo commands:

```
sudo nmap -sn -PE 10.10.1.0/24
sudo nmap -sn -PS80,443 -PA80 10.10.1.11
```

Sample output: see section 3.5 for a full line-by-line reading of both a single-host and a ping-sweep result.

Common mistakes: forgetting -sn and letting a "discovery only" scan also run a full port scan; assuming -Pn and -sn do opposite things when they are actually unrelated (-Pn skips discovery and scans anyway, -sn does discovery only and never scans ports).

What the exam asks: matching each -P letter to its probe type (see the memory trick in 3.5); recognizing that -sn means "no port scan", not "SYN".

5.2 Nmap Port Scanning Flags

Purpose: Determine which ports on a live host are open, closed, or filtered, using a chosen probe type.

```
nmap -sT|-sS|-sX|-sF|-sN|-sM|-sA|-sW|-sU|-sY|-sZ [-p PORTS] \
  [-6] TARGET
nmap -Pn -p- -sI ZOMBIE TARGET
```

Flag	Meaning	When to use	Example
-sT	TCP connect (full open) scan	No root, or want full logging	nmap -sT 10.10.1.11
-sS	TCP SYN (half-open) scan	Default root scan, quieter	sudo nmap -sS 10.10.1.11
-sX	Xmas scan (FIN+PSH+URG)	Test simple non-stateful filters	sudo nmap -sX 10.10.1.11
-sF	FIN scan	Same idea as Xmas, one flag	sudo nmap -sF 10.10.1.11
-sN	NULL scan	Same idea, no flags at all	sudo nmap -sN 10.10.1.11
-sM	Maimon scan (FIN+ACK)	BSD-derived targets	sudo nmap -sM 10.10.1.11
-sA	ACK scan	Map firewall rule behavior	sudo nmap -sA 10.10.1.11
-sW	Window scan	Refine ACK scan on some OSes	sudo nmap -sW 10.10.1.11
-sU	UDP scan	DNS, SNMP, TFTP and other UDP	sudo nmap -sU 10.10.1.11
-sY	SCTP INIT scan	SCTP services (telephony)	sudo nmap -sY 10.10.1.11
-sZ	SCTP COOKIE ECHO scan	Bypass simple INIT-only filters	sudo nmap -sZ 10.10.1.11
-sI ZOMBIE	Idle scan through a zombie host	Hide the real scanner's IP	sudo nmap -sI 10.10.1.5 TGT
-6	Scan an IPv6 target	Target has an IPv6 address	nmap -6 2001:db8::10
-p PORTS	Choose which ports to scan	Narrow or widen the scan	-p 1-1000 or -p 80,443
-p-	Scan all 65535 ports	Full sweep	nmap -p- 10.10.1.11

Flag	Meaning	When to use	Example
--top-ports N	Scan the N most common ports	Faster broad coverage	--top-ports 100

Demo commands:

```
sudo nmap -sS -p 1-1000 10.10.1.11
sudo nmap -sU --top-ports 20 10.10.1.11
sudo nmap -Pn -p- -sI 10.10.1.5 10.10.1.11
```

Sample output: see section 3.6 for full line-by-line readings of SYN, Xmas/FIN/NULL (Linux and Windows targets), ACK, window, UDP, and idle scan results.

Common mistakes: running -sX, -sF, -sN, or -sM against a Windows target and expecting useful results (Windows answers RST to everything, so all ports show closed); forgetting that UDP scanning is slow and should usually be limited with -p or --top-ports; mixing up -sA (never shows "open") with -sS (shows "open" directly).

What the exam asks: the exact flags set by each scan (Xmas = FIN+PSH+URG, Maimon = FIN+ACK); which scan type a given description matches; why FIN/NULL/Xmas scans fail against Windows.

5.3 Nmap Service and OS Detection

Purpose: Identify the exact software version behind each open port, and guess the target's operating system.

```
nmap -sV [--version-intensity 0-9] TARGET
nmap -O TARGET
nmap -A TARGET
```

Flag	Meaning	When to use	Example
-sV	Detect service name and version	After finding open ports	nmap -sV 10.10.1.11
--version-intensity N	How hard to probe (0 light, 9 max)	Balance speed vs accuracy	--version-intensity 9
--version-light	Shortcut for intensity 2	Fast, less accurate	--version-light
--version-all	Shortcut for intensity 9	Slow, most accurate	--version-all
-O	Active OS fingerprinting	Guess the target's OS	sudo nmap -O 10.10.1.11
-A	-sV + -O + default scripts + traceroute	One command, full picture	sudo nmap -A 10.10.1.11

Demo commands:

```
nmap -sV 10.10.1.11
sudo nmap -A 10.10.1.11
```

Sample output: see section 3.7 for the full VERSION-column reading, and section 3.13 for the -O/-A "Running" and "OS details" reading.

Common mistakes: running -sV or -O together with fragmentation (-f) or other raw-packet-distorting options, which Nmap itself warns breaks these features; expecting -O to give a certain answer rather than a best guess with a confidence percentage.

What the exam asks: -sV finds software versions (to match exploits); -O guesses the OS; -A bundles both plus scripts and traceroute into one aggressive scan.

5.4 Nmap Evasion Flags

Purpose: Change how a scan's packets look, to get past a firewall or IDS, or to hide the scanner's identity.

```
nmap -f | --mtu N TARGET
nmap -g PORT | --source-port PORT TARGET
nmap -D RND:N | decoy1,decoy2,...[,ME] TARGET
nmap --spoof-mac MAC|PREFIX|VENDOR TARGET
nmap --randomize-hosts TARGET_RANGE
nmap --badsum TARGET
nmap --ip-options "R|T|L addr...|S addr..." TARGET
nmap --proxies URL[,URL...] TARGET
```

Flag	Meaning	When to use	Example
-f	Fragment packets (8 bytes/fragment)	Simple filter inspects whole packets only	sudo nmap -f -sS 10.10.1.11
--mtu N	Custom fragment size (multiple of 8)	Finer control than -f	--mtu 24
-g PORT	Force a specific source port	Firewall trusts that port	sudo nmap -g 80 10.10.1.11
--source-port PORT	Same as -g, long form	Same as -g	--source-port 53
-D RND:N	N random decoy source IPs	Hide scanner among random noise	sudo nmap -D RND:10 10.10.1.11
-D list[,ME]	Manually chosen decoy IPs	Control exactly which decoys	-D 10.1.1.5,10.1.1.9,ME
--spoof-mac VALUE	Fake source MAC (needs raw scan)	Test MAC-based trust (local LAN)	--spoof-mac Dell
--randomize-hosts	Shuffle scan order across a range	Avoid an obvious sequential sweep	--randomize-hosts
--badsum	Send invalid TCP/UDP/SCTP checksums	Test if a device answers anyway	--badsum
--ip-options "..."	Set IP header options, incl. source routing	Route around a firewalled router	--ip-options "L 10.1.1.1"
--proxies URL,...	Relay NSE/version traffic via proxies	Hide true source for those features	--proxies http://10.1.1.5:8080

Demo commands:

```
sudo nmap -f -sS 10.10.1.11
sudo nmap -g 80 -sS 10.10.1.11
sudo nmap -D RND:10 10.10.1.11
sudo nmap -sT -Pn --spoof-mac Dell 10.10.1.11
sudo nmap --randomize-hosts 10.10.1.0/24
sudo nmap --badsum 10.10.1.11
```

Sample output: see sections 3.9 through 3.18 for full, line-by-line readings of each of these flags in action.

Common mistakes: pairing --spoof-mac with -sT, which Nmap will not honor (needs a raw-packet scan type such as -sS); expecting --badsum to ever show "open" from the real target (a correctly built stack always drops a bad-checksum packet); using decoys that could not plausibly be on the attacker's network path.

What the exam asks: matching each evasion flag to its one-line description (fragment, source port, decoy, MAC spoof, randomize order, bad checksum, source route, proxy); recognizing the specific limitation shown in the --spoof-mac plus -sT screenshot.

5.5 Hping3

Purpose: Craft and send custom TCP, UDP, ICMP, or raw IP packets, including spoofed-source packets, for scanning, testing firewall rules, or simple denial-of-service testing in a lab.

```
hping3 [-c count] [-S|-A|-F|-R|-P|-U] [-p port] \
  [-a spoofed_ip] [-t ttl] [--scan ports] TARGET
```

Flag	Meaning	When to use	Example
-c, --count N	Number of packets to send	Limit how many probes go out	-c 5
-S	Set the SYN flag	Simple SYN-style probe	hping3 -S -p 80 TARGET
-A	Set the ACK flag	ACK-style probe	hping3 -A -p 80 TARGET
-F	Set the FIN flag	FIN-style probe	hping3 -F -p 80 TARGET
-1, --icmp	ICMP mode (echo request by default)	Ping-like test, custom ICMP	hping3 -1 TARGET
-2, --udp	UDP mode	UDP-based probing	hping3 -2 -p 53 TARGET
-p, --baseport/-p	Destination port for the probe	Choose the target port	-p 80
-a, --spoof HOST	Spoof the source IP address	Hide the real sender (3.11)	hping3 TARGET -a 7.7.7.7
-t, --ttl N	Set the outgoing TTL	Combine with --traceroute	-t 1 --traceroute
--traceroute	Run in traceroute mode	Map the path to a target	hping3 --traceroute -1 TARGET

Flag	Meaning	When to use	Example
-8, --scan ports	Scan mode across a list of ports	Act like a simple port scanner	--scan 1-1000,8888 -S TARGET
-V, --verbose	Show detailed reply info (TTL, win, id)	Read extra diagnostic detail	-V

Demo commands:

```
hping3 www.certifiedhacker.com -a 7.7.7.7
sudo hping3 -c 5 -S -p 80 -a 7.7.7.7 www.certifiedhacker.com
sudo hping3 --scan 1-1000,8888 -S 10.10.1.11
```

Sample output: see section 3.11 for a reading of the SYN+spoofed-source example and the reasoning behind why the real reply is never seen by the attacker in a basic spoof.

Common mistakes: expecting to see the victim's reply after spoofing the source with -a (the reply goes to the spoofed address, not back to you, as explained in 3.11); forgetting that --scan still honors the chosen flag (-S, and so on), so you must still pick a probe type even in scan mode.

What the exam asks: that -a (or --spoof) is the Hping3 flag for IP spoofing; that spoofed scans cannot complete a real TCP handshake; that --scan turns Hping3 into a basic port scanner while keeping its crafting flexibility.

5.6 Nmap Output and Timing Options (ADDED CONTEXT)

Purpose: Control how fast a scan runs and how its results are saved, independent of which scan type is chosen.

```
nmap -T0|1|2|3|4|5 TARGET
nmap -oN file.txt | -oX file.xml | -oG file.gnmap | -oA basename TARGET
nmap -iL targets.txt
nmap -n TARGET
```

Flag	Meaning	When to use	Example
-T0 to -T5	Timing template, 0 slowest/stealthiest to 5 fastest	Balance speed vs. detection risk	-T2
-oN file	Save normal (human-readable) output	Keep a readable log	-oN scan.txt
-oX file	Save XML output	Feed into other tools or reports	-oX scan.xml
-oG file	Save "grepable" output	Quick command-line parsing	-oG scan.gnmap
-oA basename	Save all three formats at once	Convenience	-oA scan

Flag	Meaning	When to use	Example
-iL file	Read target list from a file	Scan many hosts from a list	-iL targets.txt
-n	Skip DNS resolution	Speed up large scans	-n
-v, -vv	Increase verbosity (one or two levels)	See more detail while running	-vv

Demo command:

```
sudo nmap -sS -T2 -n -oA client_scan -iL targets.txt
```

Common mistakes: choosing a very fast timing template (-T4 or -T5) on a fragile or low-bandwidth network, causing dropped probes that look like filtering; forgetting that -T0 and -T1 can make a large scan take hours or days.

What the exam asks: that -T controls speed and stealth trade-off, not scan type; which output flag produces which file format.

5.7 Reading OS Clues in Wireshark and tcpdump (ADDED CONTEXT)

Purpose: Passively read TTL and TCP window size from a captured packet to guess the target OS (section 3.8), without sending any extra probe.

```
tcpdump -nn -i eth0 host 10.10.1.11 and tcp
```

Field (Wireshark path)	Meaning	Example value	Clue
Internet Protocol Version 4 > Time to live	IP header TTL	128	Likely Windows
Transmission Control Protocol > Window size value	TCP header window	5840	Likely Linux
[Calculated window size]	Window after scaling	65535	Confirms scaling is active

Demo (tcpdump, flag letters):

```
10.10.1.5.51514 > 10.10.1.11.80: Flags [S], seq 10, win 64240
10.10.1.11.80 > 10.10.1.5.51514: Flags [S.], seq 1, ack 11,
  win 8192, ttl 128
```

How to read it: [S] = SYN only, [S.] = SYN+ACK, [.] = ACK only, [F] = FIN, [R] = RST, [P] = PSH, [U] = URG. The second line's "ttl 128" and "win 8192" match the Windows row in the OS table from section 3.8.

Common mistakes: reading the TTL of the wrong packet (the client's own outgoing packet, not the target's reply); forgetting that TTL decreases with hop count, so a captured value may be slightly below the OS default.

What the exam asks: matching a given TTL/window pair to the closest OS in the reference table from section 3.8.

5.8 Netcat and Telnet Banner Grabbing (ADDED CONTEXT)

Purpose: Manually connect to a service port and read whatever banner it sends first, a simple form of active banner grabbing (section 3.13).

```
nc -nv TARGET PORT
telnet TARGET PORT
```

Flag	Meaning	When to use	Example
-n (nc)	Skip DNS resolution	Speed, or target has no DNS entry	nc -nv 10.10.1.11 21
-v (nc)	Verbose, show connection status	Confirm the connection opened	nc -nv 10.10.1.11 80

Demo and sample output:

```
nc -nv 10.10.1.11 21
```

```
(UNKNOWN) [10.10.1.11] 21 (ftp) open
220 Microsoft FTP Service
```

How to read it: the first line confirms Netcat opened a raw TCP connection to port 21. The second line is the banner the FTP service sent immediately, unprompted, naming the exact product.

Common mistakes: expecting every service to send a banner immediately (some, such as a plain HTTP server, only reply after you send a request, for example GET / HTTP/1.0 followed by two newlines); treating a banner as absolute proof rather than a clue (banners can be edited or faked by a defender).

What the exam asks: that this manual technique is a simple, direct form of active banner grabbing, distinct from the TTL/window method (passive, section 3.8) and from reading error pages or URL extensions (passive, section 3.13).

6. Ports, Protocols, Standards and Acronyms

6.1 Ports Seen in This Module

Port	Service	Simple purpose	Weakness attackers look for
21/tcp	ftp	File transfer, control channel	Cleartext login, old server versions
53	domain (DNS)	Name resolution	Open UDP ping/scan target, zone data
80/tcp	http	Web traffic	Reveals web server and OS via banners
135/tcp	msrpc	Microsoft RPC endpoint mapper	Entry point to many Windows services
139/tcp	netbios-ssn	NetBIOS session service	Legacy Windows file/ print sharing
161	snmp	Device monitoring and management	Weak/default community strings
443/tcp	https	Encrypted web traffic	Still fingerprinted via TLS/cert data
445/tcp	microsoft-ds	SMB file sharing	Shares, known SMB exploits
1900	ssdp	UPnP device discovery	Device enumeration, DoS reflection
3389/tcp	ms-wbt-server	Remote Desktop Protocol (RDP)	Exposed remote admin access

6.2 Protocol Layer Reference

Protocol	Simple purpose	Layer	Known weakness	Related tool/ technique
TCP	Reliable, connection-based transport	Transport	Handshake and flags can be probed	Nmap -sT/-sS/-sX/-sF/-sN/-sM/-sA/-sW
UDP	Fast, connectionless transport	Transport	No handshake, slow to scan fully	Nmap -sU
ICMP	Error and control messages	Network	Reveals live hosts and routers	Ping scans (3.5), traceroute
ARP	Resolves IP to MAC on a LAN	Link	Always answered, hard to block	ARP ping (-PR, 3.5)
SCTP	Reliable transport for telephony signalling	Transport	INIT/COOKIE chunks probed like TCP flags	Nmap -sY/-sZ

Protocol	Simple purpose	Layer	Known weakness	Related tool/ technique
SSDP/UPnP	Device discovery over multicast	Application	Open by default on many devices	SSDP scanning, reflection DoS
IPv6	Next-generation addressing (128-bit)	Network	Huge space, less filtered, no ARP (uses ND)	Nmap -6

6.3 TCP Flags Quick Reference

Flag	Full name	One-line meaning
URG	Urgent	Process this data immediately
ACK	Acknowledgement	Confirms receipt of data
PSH	Push	Deliver buffered data now
RST	Reset	Abruptly ends a connection
SYN	Synchronize	Starts a new connection
FIN	Finish	Politely ends a connection

6.4 ICMP Types Used in Scanning

Type	Name	Used for
0	Echo reply	Answer to a ping
3	Destination unreachable	Code 3 = port unreachable (closed UDP port)
8	Echo request	Ping probe (-PE)
11	Time exceeded	Traceroute hop discovery
13 / 14	Timestamp request / reply	-PP host discovery
17 / 18	Address mask request / reply	-PM host discovery

6.5 Rare Protocols, Short Introductions

- **SCTP (Stream Control Transmission Protocol):** a transport protocol, IP protocol number 132, built for telephony signalling (SS7 over IP) and used in some mobile core network interfaces. Network engineers in telecom use it. Attackers care because it has its own four-way handshake (INIT, INIT-ACK, COOKIE-ECHO, COOKIE-ACK) that can be probed the same way TCP's three-way handshake is probed, and because it is less commonly filtered than TCP or UDP, so it can slip past firewalls tuned only for the two common transports.
- **SSDP (Simple Service Discovery Protocol) and UPnP (Universal Plug and Play):** SSDP is the discovery half of UPnP, running over UDP port 1900 with multicast address 239.255.255.250. Home and office users rely on it so that routers, smart TVs, cameras, and printers can find each other automatically. Attackers care because it often runs with no authentication by default, it reveals

the make and model of devices through M-SEARCH replies, and it has been abused as a reflection and amplification vector in denial-of-service attacks (covered further in Module 10).

- **ICMP timestamp and address mask queries:** these two ICMP query types were designed to let a host ask for the current time or its own subnet mask from another host. Few systems answer them meaningfully today, but administrators who remember to block ICMP echo often forget these two, so a reply to either one still proves a host is alive (section 3.5).

6.6 Acronyms

Acronym	Meaning
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
ICMP	Internet Control Message Protocol
ARP	Address Resolution Protocol
SCTP	Stream Control Transmission Protocol
SSDP	Simple Service Discovery Protocol
UPnP	Universal Plug and Play
TTL	Time To Live
MTU	Maximum Transmission Unit
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
OUI	Organizationally Unique Identifier
MAC	Media Access Control (hardware address)
ISN	Initial Sequence Number
ACL	Access Control List
NAC	Network Access Control
CPE	Common Platform Enumeration
NSE	Nmap Scripting Engine
RDP	Remote Desktop Protocol
SMB	Server Message Block
RPC	Remote Procedure Call
SOC	Security Operations Center
BCP 38	Best Current Practice 38, network ingress filtering standard

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Scanning Technique vs Countermeasure

Scanning technique	What it achieves	Best countermeasure
ICMP/ARP/UDP/TCP/protocol ping scans	Finds which hosts are alive	Filter ICMP types, limit ARP to local segment, stateful firewall
TCP connect scan (-sT)	Finds open ports, fully logged	Firewall/IDS rules, update firmware
TCP SYN scan (-sS)	Finds open ports, quieter	Same, plus detect half-open connection floods
Xmas, FIN, NULL, Maimon scans	Try to slip past simple filters	Stateful inspection, not simple flag-based rules
ACK and window scans	Map firewall rule behavior	Anti-scanning rules, stateful firewall that drops stray ACKs
UDP scan	Finds open UDP services	Rate-limit ICMP unreachable, block unused UDP ports
Idle scan (zombie)	Hides the real scanner's IP	Anti-spoofing rules, egress filtering
Packet fragmentation	Hides probe from simple filters	Reassemble before inspecting (stateful FW/IDS)
Source routing	Routes around a firewalled router	Disable IP source routing on routers
Source port manipulation	Exploits port-based trust rules	Never trust by source port alone
IP address decoy	Hides the real scanner among fakes	Anti-scanning rules, routing plausibility checks
IP address spoofing	Hides or fakes the sender's identity	Ingress/egress filtering, random ISN
MAC address spoofing	Fakes the sender's hardware identity	802.1X, switch port security
Creating custom packets	Avoids a tool-signature-based filter	Behavior-based detection, not signatures
Randomizing host order	Avoids an obvious sequential pattern	Detect scan behavior, not sequence alone
Sending bad checksums	Tests if a device answers without checking	Ensure devices properly validate checksums
Proxy servers / anonymizers	Hides scanner behind another relay	Block known relay ranges, behavior detection

7.2 Look-alike Pairs

X vs Y	Difference in one line	How to tell in a question
Footprinting vs scanning	Collect public info vs send probes to the target	"Sent a packet" or "ran Nmap" = scanning
-sn vs -Pn	-sn = ping only, no port scan; -Pn = skip ping, scan anyway	Which step does the flag skip?
-PS vs -sS	-PS = TCP SYN host discovery probe; -sS = TCP SYN port scan	"Host discovery" vs "port scan" in the question
-PU vs -sU	-PU = UDP host discovery probe; -sU = UDP port scan	Same pattern as above
-sT vs -sS	Connect = full handshake, no root; SYN = half-open, needs root	"Does not complete the handshake" = SYN
-sX/-sF/-sN vs -sM	Xmas/FIN/NULL use no-reply = open; Maimon's probe is FIN+ACK	Which exact flags are set
-sA vs -sW	ACK never shows "open"; Window reads RST window field	"Never determines open" = ACK alone
-sS vs -sY	TCP SYN scan vs SCTP INIT scan, same idea on different protocols	Check which transport protocol is named
IP spoofing vs IP decoy	Spoofing replaces the real source; decoy adds fakes beside it	"Cannot see reply" = spoofing; "hidden in a crowd" = decoy
IP spoofing vs MAC spoofing	IP fakes the routed address; MAC fakes the local link	Is the field an IP or a hardware address?
Source routing vs source port manipulation	Routing controls the path; port manip. fakes the sender's port	"Route around a router" vs "pretend to be port 80"
Active vs passive banner grabbing	Active crafts packets; passive reads errors or traffic	"Crafted packets" = active; "error" = passive
Ingress filtering vs egress filtering	Ingress checks traffic IN; egress checks traffic OUT	Which direction does the question describe?
Proxy server vs anonymizer	A proxy is one relay; an anonymizer is a wider network	Is it one relay or a whole hiding network?

8. Exam Cheat Sheet

8.1 Keyword -> Answer

If the question says	Think
"identify hosts, ports, and services"	Network scanning (phase 2)
"no further transmissions"	FIN flag
"processed immediately"	URG flag
"acknowledges receipt"	ACK flag
"sends buffered data immediately"	PSH flag
"resets a connection"	RST flag
"initiates a connection"	SYN flag
"ARP request/reply, local LAN, most reliable"	ARP ping (-PR)
"ICMP timestamp or address mask query"	-PP / -PM host discovery
"ping scan only, no port scan"	-sn
"skip host discovery"	-Pn
"full TCP handshake completed, no root needed"	TCP connect scan (-sT)
"half-open, does not complete handshake"	TCP SYN scan (-sS)
"FIN + PSH + URG"	Xmas scan (-sX)
"no flags at all"	NULL scan (-sN)
"FIN + ACK, BSD-derived systems"	Maimon scan (-sM)
"never shows open, maps firewall rules"	ACK scan (-sA)
"reads the TCP window of a RST reply"	Window scan (-sW)
"uses a zombie host and its IP ID"	Idle scan (-sI)
"ICMP port unreachable means closed"	UDP scan (-sU)
"INIT / INIT-ACK / ABORT"	SCTP INIT scan (-sY)
"detects service name and version"	-sV
"guesses the operating system"	-O (or -A for the full bundle)
"TTL and TCP window size of the first packet"	Passive OS guess (section 3.8)
"specially crafted packets compared to a database"	Active banner grabbing
"error messages, sniffed traffic, or URL extension"	Passive banner grabbing
".aspx in the URL"	IIS server, Windows platform
"split the probe into small IP fragments"	Packet fragmentation (-f)
"originator dictates the route"	Source routing (--ip-options)
"firewall trusts a well-known source port"	Source port manipulation (-g)

If the question says	Think
"decoys mixed with the real scanner"	IP address decoy (-D)
"attack appears to come from someone else, cannot see the reply"	IP address spoofing (-a in Hping3)
"fake hardware address, needs raw socket"	MAC address spoofing (--spoof-mac)
"build a packet field by field"	Packet crafting tool (Colasoft, NetScanTools Pro)
"scan hosts out of sequence"	Randomizing host order (--randomize-hosts)
"deliberately invalid checksum"	Sending bad checksums (--badsum)
"relay through another server"	Proxy server
"hide identity through a broader network"	Anonymizer (Tor)
"checks traffic coming into your network"	Ingress filtering
"checks traffic leaving your network"	Egress filtering
"do not trust a connection by address alone"	No IP-based authentication

8.2 Must-memorize Lists and Numbers

- Five-phase model position: scanning is phase 2 (after reconnaissance, before gaining access).
- TCP flags: 6 (URG, ACK, PSH, RST, SYN, FIN).
- Three-way handshake: 3 packets (SYN, SYN+ACK, ACK). Termination: 4 packets (FIN, ACK, FIN, ACK).
- Host discovery flags: 8 (-PE, -PP, -PM, -PR, -PU, -PS, -PA, -PO), plus -sn and -Pn as modifiers.
- Port scan flags: -sT, -sS, -sX, -sF, -sN, -sM, -sA, -sW, -sI, -sU, -sY, -sZ, -6 (13 named scan-related flags).
- OS TTL defaults: Linux/FreeBSD 64; Windows 128; Cisco/Solaris/OpenBSD/AIX 255.
- Evasion techniques beyond IDS and firewall: 10 (fragmentation, source routing, source port manipulation, IP decoy, IP spoofing, MAC spoofing, custom packets, randomize host order/bad checksums counted together or separately per source, proxy servers, anonymizers).
- Port scanning countermeasures: 8 items (section 3.19).
- IP spoofing countermeasures: 6 items (section 3.20).
- Well-known ports seen: 21, 80, 135, 139, 445, 3389 (plus 53, 161, 1900 in added context).

8.3 "Pick the BEST option" Rules for This Module

1. If a question stresses "does not complete the handshake", the answer is the SYN (half-open) scan, not the connect scan.
2. If a question stresses "cannot tell open from closed, only filtered from unfiltered", the answer is the ACK scan.
3. If a question stresses "fails against Windows", the answer is one of FIN, NULL, Xmas, or Maimon, because Windows answers RST to all of them.

4. If a question stresses "hides the real scanner's IP from the target entirely while still getting a usable result", the answer is the idle scan (decoys only add noise; spoofing alone sacrifices the reply).
5. If a question stresses "a reply came back that should not have, from a broken packet", the answer points to a firewall or IDS answering on the target's behalf (bad checksum test).
6. If a question stresses "best first step to confirm an open port is actually vulnerable", the answer is service version detection (-sV), not just the open/closed state.
7. For countermeasure questions, favor the option that fixes the root trust assumption (do not trust by source port, IP, or MAC alone) over an option that only reacts after the fact.

9. Practice Questions

These are original practice questions written for this guide. They are not real exam questions.

Q1. During a TCP three-way handshake, a client sends a SYN packet with sequence number 500. What acknowledgement number will the server place in its SYN+ACK reply?

- A. 500
- B. 501
- C. 0
- D. 1000

Answer: B. The acknowledgement number equals the received sequence number plus 1, so the server acknowledges 501.

- A is wrong: that would mean the server did not actually acknowledge receipt of the byte.
- C is wrong: 0 would only make sense before any sequence number was exchanged.
- D is wrong: there is no rule that doubles the sequence number.

Q2. Which TCP flag tells the receiver to process the data in the packet immediately, rather than waiting in a buffer?

- A. PSH
- B. URG
- C. ACK
- D. SYN

Answer: B. URG marks data that should be treated as urgent and processed right away. (PSH tells the stack to flush buffered data to the application immediately, a related but distinct meaning: PSH is about delivery timing, URG is about priority within the stream.)

- A is close but describes pushing buffered data out, not marking data as urgent.
- C is wrong: ACK only confirms receipt.
- D is wrong: SYN starts a connection.

Q3. An administrator notices that ICMP echo requests are blocked at the firewall, but a security tool still manages to discover live hosts using ICMP by asking for the current time from each host. Which Nmap option is this?

- A. -PE
- B. -PP
- C. -PR
- D. -PO

Answer: B. -PP sends an ICMP timestamp request, asking for the current time; a reply (ICMP type 14) proves the host is alive even when echo is blocked.

- A is wrong: -PE is the blocked echo request.
- C is wrong: -PR is an ARP ping, not ICMP.
- D is wrong: -PO sends IP protocol pings, not a timestamp query.

Q4. Which Nmap host discovery flag tells Nmap to skip port scanning entirely and only report which hosts are alive?

- A. -Pn
- B. -sn
- C. -sS
- D. -O

Answer: B. -sn runs only the ping/discovery phase and stops before scanning any ports.

- A is wrong: -Pn does the opposite, it skips discovery and scans ports anyway.
- C is wrong: -sS is a port scan type.
- D is wrong: -O is OS detection.

Q5. A scanner sends a TCP packet with only the FIN flag set to a Linux target. The port does not respond at all. According to the FIN scan's logic, what does this mean?

- A. The port is closed
- B. The port is open or filtered
- C. The port does not exist
- D. The host is down

Answer: B. For FIN, NULL, Xmas, and Maimon scans, silence means the port is open or filtered; only a RST reply would mean closed.

- A is wrong: closed ports answer with RST in this scan type.
- C is wrong: ports always "exist" as numbers; the question is whether something listens on it.
- D is wrong: the host clearly processed the probe (or a firewall near it did); this alone does not prove it is down.

Q6. The same FIN scan from Q5 is run against a Windows Server target instead, and every single port shows closed. What is the most likely explanation?

- A. The firewall is extremely well configured
- B. Windows replies with RST to this probe regardless of the real port state
- C. The FIN scan always fails on any OS
- D. The target has no open ports at all

Answer: B. Microsoft Windows, along with many Cisco devices, BSDI, and IBM OS/400, violates the RFC 793 expectation and sends RST for FIN/NULL/Xmas probes on every port, open or closed, which makes this scan type unreliable on those systems.

- A is wrong: this is a known OS behavior difference, not evidence of firewall quality.
- C is wrong: the scan does work, imperfectly, on many Unix-based systems.
- D is an unsupported guess; the real cause is the known OS-specific RST behavior.

Q7. Which scan type is specifically designed to determine whether firewall rules are stateful, and by itself can never report a port as "open"?

- A. SYN scan
- B. ACK scan
- C. UDP scan
- D. Idle scan

Answer: B. The ACK scan's probe has only the ACK flag; both open and closed ports answer RST, so the scan can only report "unfiltered" or "filtered", never "open".

- A is wrong: SYN scan directly reports open ports via SYN/ACK replies.
- C is wrong: UDP scan can report open directly if a UDP reply comes back.
- D is wrong: idle scan, through the zombie's IP ID behavior, does distinguish open from closed/filtered.

Q8. An attacker wants to scan a target without the target's logs ever showing the attacker's own IP address, while still being able to determine which ports are open. Which technique BEST fits this goal?

- A. IP address decoy
- B. Idle scan using a zombie host
- C. Randomizing host order
- D. Sending bad checksums

Answer: B. The idle scan spoofs every probe as coming from the zombie, and the attacker reads the result indirectly through the zombie's IP ID counter, so the target never logs the attacker's real address while a usable result is still obtained.

- A is wrong: a decoy scan still includes the real scanner's own IP among the decoys; it only adds noise, it does not fully hide it.
- C is wrong: randomizing order changes the sequence of hosts scanned, not whether the attacker's IP appears.
- D is wrong: bad checksums test firewall/IDS behavior, unrelated to hiding the scanner's identity.

Q9. A tester captures a reply packet from a target with TTL 64 and a TCP window size of 65535. According to the module's reference table, which operating system is the BEST match?

- A. Linux
- B. FreeBSD
- C. Windows
- D. Cisco Router

Answer: B. TTL 64 fits both Linux and FreeBSD, but the window size of 65535 specifically matches FreeBSD; Linux's default in the table is 5840.

- A is wrong: Linux's listed default window size is 5840, not 65535.
- C is wrong: Windows uses TTL 128, not 64.
- D is wrong: Cisco routers use TTL 255.

Q10. Which of the following is an example of PASSIVE banner grabbing, as described in this module?

- A. Sending a series of specially crafted packets and comparing the responses to a fingerprint database
- B. Reading the server name printed in a web application's error page
- C. Running Nmap's -O active OS detection
- D. Sending a TCP SYN probe and reading the SYN/ACK flags

Answer: B. Reading information that a target reveals on its own, such as in an error message, without sending any special crafted probe, is passive banner grabbing.

- A is wrong: this describes active banner grabbing.
- C is wrong: -O is an active fingerprinting technique.
- D is wrong: this is a normal scan probe, not a banner-grabbing technique at all.

Q11. Which URL file extension, mentioned specifically in this module, suggests that a web application is running on Microsoft IIS and a Windows platform?

- A. .php
- B. .jsp
- C. .aspx
- D. .cfm

Answer: C. The module names .aspx => IIS server and Windows platform as its specific example of banner grabbing from page extensions.

- A is wrong: .php commonly suggests Apache or Nginx, on Linux or Windows, but is not the example given.
- B is wrong: .jsp suggests a Java application server such as Tomcat.
- D is wrong: .cfm suggests Adobe ColdFusion.

Q12. A scan of a firewall-protected network returns all ports filtered with a plain SYN scan. The tester then runs the scan again, splitting each probe into several small IP fragments. Several ports now show open. What technique did the tester use?

- A. Source routing
- B. IP address decoy
- C. Packet fragmentation
- D. MAC address spoofing

Answer: C. Splitting a probe packet into several smaller IP fragments so a simple filter cannot see the complete header is the definition of packet fragmentation.

- A is wrong: source routing changes the path a packet takes through routers, not its fragmentation.
- B is wrong: IP address decoy adds fake source addresses; it does not split the packet.
- D is wrong: MAC address spoofing changes the hardware address field, not the packet's fragmentation.

Q13. A firewall is configured to allow any traffic that appears to originate from port 53, assuming it must be legitimate DNS traffic. Which Nmap option lets an attacker exploit this specific misconfiguration during a scan?

- A. --badsum
- B. -g 53 (or --source-port 53)
- C. -D RND:10
- D. --spoof-mac

Answer: B. -g (or --source-port) forces Nmap to use a chosen source port, here 53, to match the firewall's trusted-port rule.

- A is wrong: --badsum sends invalid checksums, unrelated to the source port.
- C is wrong: -D adds decoy source IP addresses, not a fixed source port.
- D is wrong: --spoof-mac changes the hardware address, not the TCP source port.

Q14. Which statement correctly explains why an attacker generally cannot read the direct reply to a scan after spoofing the source IP address with Hping3's -a option?

- A. Hping3 automatically discards all replies when -a is used
- B. The target's reply is sent to the spoofed address, not to the attacker's real address
- C. Spoofed packets are always dropped by the target before any reply is generated
- D. The -a option disables all TCP flags in the outgoing packet

Answer: B. Because the packet's source address field says the spoofed IP, the target routes its reply back to that spoofed address rather than to the attacker's real location, so the attacker normally never sees it directly.

- A is wrong: Hping3 does not discard replies on purpose; it simply never receives the real one at its own interface.
- C is wrong: the target generally does process the spoofed packet and does reply, just not to the sender's real address.
- D is wrong: -a only changes the source address field, not the TCP flags chosen with other options.

Q15. A tester runs the command below and sees a warning that the requested option "will not be honored for TCP Connect scan." What is the most likely cause?

```
sudo nmap -sT -Pn --spoof-mac Dell 10.10.1.11
```

- A. -Pn is incompatible with --spoof-mac
- B. --spoof-mac requires a raw-packet scan type, and -sT uses the OS's own socket interface instead
- C. The vendor name "Dell" is not a valid value for --spoof-mac
- D. 10.10.1.11 is not a valid IPv4 address

Answer: B. MAC address spoofing needs Nmap to build raw Ethernet frames itself; -sT (TCP connect scan) instead relies on the operating system's normal connect() call, which does not let Nmap set a custom MAC address, producing exactly this warning.

- A is wrong: -Pn (skip host discovery) has no bearing on this specific conflict.
- C is wrong: "Dell" is a perfectly valid vendor name that Nmap can match to an OUI.
- D is wrong: 10.10.1.11 is a valid IPv4 address and is not the source of the warning.

Q16. Which of the following BEST distinguishes IP address decoy scanning from IP address spoofing, as described in this module?

- A. Decoy scanning never works against Windows targets, while spoofing always does
- B. Decoy scanning hides the real scanner's IP among several fake ones while still letting the attacker read a usable result; spoofing replaces the real source entirely and generally prevents the attacker from seeing the reply
- C. Decoy scanning only affects UDP scans, while spoofing only affects TCP scans
- D. They are two names for exactly the same technique

Answer: B. This is the key practical difference highlighted in sections 3.11 and 3.16: a decoy scan keeps the real scanning IP in the mix of apparent sources, so the attacker

still gets the actual scan result, while a fully spoofed single source address means the reply goes elsewhere.

- A, C, and D all misstate the relationship; neither technique is limited to one OS or one transport protocol, and they are distinct techniques with different trade-offs.

Q17. A tester runs an Nmap scan with --badsum against a target known to have several open ports, and every single port comes back "filtered (no-response)". What is the correct interpretation?

- A. The target has no open ports
- B. The target's TCP/IP stack is behaving correctly by dropping packets with invalid checksums
- C. --badsum always reports open ports accurately regardless of checksum validity
- D. The firewall has crashed

Answer: B. A properly implemented TCP/IP stack validates the checksum and silently drops a packet with an invalid one, so "filtered (no-response)" for every port is the expected, correct behavior, not evidence of a flaw.

- A is wrong: the module's own screenshot shows this exact host has several genuinely open ports under a normal scan.
- C is wrong: --badsum specifically relies on an invalid, broken packet, so it cannot reliably show "open".
- D is an unsupported leap; nothing here indicates a crash.

Q18. Which countermeasure from this module's Port Scanning Countermeasures list specifically targets the technique of using a trusted source port or a specified source route to bypass filtering?

- A. "Use a custom rule set to lock down the network and block unwanted ports at the firewall"
- B. "Ensure that the mechanisms used for routing and filtering at the routers and firewalls, respectively, cannot be bypassed using a particular source port or source routing methods"
- C. "Filter all ICMP messages at the firewalls and routers"
- D. "Ensure that the router, IDS, and firewall firmware are updated to their latest releases and versions"

Answer: B. This is the countermeasure that directly names both source port and source routing bypass methods.

- A, C, and D are all real countermeasures from the same list, but none specifically names source port or source routing bypass.

Q19. Which IP spoofing countermeasure specifically stops a network from being used to launch spoofed attacks against other networks?

- A. Ingress filtering
- B. Egress filtering
- C. Random initial sequence number
- D. Multiple firewalls

Answer: B. Egress filtering drops outgoing packets whose source address is not actually part of your own address space, preventing your network from sending forged-source traffic out toward the internet.

- A is wrong: ingress filtering protects your own network from incoming spoofed traffic, not the reverse.
- C is wrong: a random ISN defends against blind TCP handshake spoofing, not against your network launching spoofed traffic outward.
- D is wrong: multiple firewalls add depth of protection generally, but are not specifically an anti-spoofing-outbound control.

Q20. An attacker wants to scan a Domain Name System (DNS) server and a Simple Network Management Protocol (SNMP) agent, both of which run over UDP. Which Nmap scan flag MUST be used to properly test these services?

- A. -sS
- B. -sU
- C. -sT
- D. -sA

Answer: B. DNS (port 53) and SNMP (port 161) commonly run over UDP, so a UDP scan (-sU) is required; a TCP-only scan type would miss them.

- A, C, and D are all TCP-based scan types and will not properly probe UDP services.

Q21. In an idle (zombie) scan, the attacker probes the zombie twice: once before and once after the spoofed probe to the real target. The zombie's IP ID rises by 2 between the two probes. What does this indicate about the target port?

- A. The port is closed
- B. The port is filtered
- C. The port is open
- D. The zombie is not usable for idle scanning

Answer: C. A rise of 2 in the zombie's IP ID means the zombie sent one extra packet in between, because the target's SYN/ACK (sent only when the port is open) prompted the zombie to reply with an unsolicited RST, incrementing its counter a second time.

- A and B are wrong: a closed or filtered port causes the target to answer the zombie with RST (or nothing), which the idle zombie ignores, so the IP ID would rise by only 1 (or not change meaningfully for filtered).
- D is wrong: a rise of exactly 2 is the expected, usable signal for an open port; it indicates the zombie is behaving predictably, which is exactly what makes it usable.

Q22. An attacker sends an SCTP COOKIE ECHO chunk to a port. No reply is received at all. According to this module's logic for this scan type, what does this mean?

- A. The port is closed
- B. The port is open or filtered
- C. The association was fully established
- D. SCTP does not support this type of probe

Answer: B. Like FIN, NULL, and Xmas scans on TCP, the SCTP COOKIE ECHO scan relies on silence meaning open or filtered; only an ABORT reply indicates closed.

- A is wrong: a closed SCTP port replies with ABORT, not silence.
- C is wrong: a single COOKIE ECHO chunk with no further exchange does not complete an SCTP association.
- D is wrong: SCTP does support this probe; it is exactly the scan type this question describes.

Q23. Which network protocol, mentioned in this module as an evasion-adjacent discovery topic, runs over UDP port 1900 and is used by routers, cameras, and smart TVs to discover each other automatically?

- A. SCTP
- B. ARP
- C. SSDP
- D. ICMP

Answer: C. SSDP (Simple Service Discovery Protocol), the discovery component of UPnP, uses UDP port 1900 for this purpose.

- A is wrong: SCTP is an unrelated transport protocol used for telephony signalling.
- B is wrong: ARP resolves IP addresses to MAC addresses on a local segment, unrelated to UPnP device discovery.
- D is wrong: ICMP carries error and control messages, not UPnP device discovery traffic.

Q24. A tester cannot find a pre-built Nmap scan type that matches a specific firewall bypass she has identified, so she opens Colasoft Packet Builder and manually edits the Ethernet, IP, and TCP fields of a single packet before sending it. Which technique is this?

- A. Service version discovery
- B. Creating custom packets
- C. IP address decoy
- D. Randomizing host order

Answer: B. Building a packet field by field with a dedicated packet crafting tool, rather than relying on a scanner's built-in probe templates, is the definition of creating custom packets.

- A is wrong: service version discovery (-sV) probes already-open ports for software version strings, unrelated to hand-building a packet.
- C is wrong: IP address decoy adds fake source addresses to an otherwise normal scan.
- D is wrong: randomizing host order changes the sequence in which hosts are scanned.

Q25. A packet diagram shows a sender whose packet is blocked at a firewalled router on the direct path to the destination. The sender instead specifies an alternate path through several other routers to reach the same destination, bypassing the firewalled router entirely. Which technique does this describe?

- A. Source routing
- B. IP address spoofing
- C. MAC address spoofing
- D. Egress filtering

Answer: A. Source routing lets the sender specify some or all of the route a packet takes, bypassing routers that would otherwise inspect or block it, exactly as shown in this diagram.

- B is wrong: IP address spoofing changes the claimed sender address, not the path taken.
- C is wrong: MAC address spoofing changes the hardware address, not the routing path.
- D is wrong: egress filtering is a defensive countermeasure, not an attacker's evasion technique, and it concerns outbound address validation, not routing paths.

Q26. An ethical hacker wants an AI assistant to help automate a routine scanning task from a plain-English description. Which of the following is the SAFEST way to use an AI-generated Nmap or Hping3 command in an authorized engagement?

- A. Run any AI-generated command immediately without review, since AI tools are always accurate

- B. Read and understand every flag in the generated command, confirm it matches the agreed scope, and then decide whether to run it
- C. Only use AI-generated commands against targets outside the signed scope, to avoid liability
- D. Never use AI tools for scanning tasks under any circumstances

Answer: B. As with any tool output, an AI-generated command should be read and understood, and checked against the written scope of the engagement, before it is executed; this matches the "AI angle" guidance repeated throughout this module.

- A is wrong: AI assistants can generate incorrect, outdated, or unsafe commands, including nonexistent flags.
- C is wrong: this inverts the whole point of scope; targets outside the signed scope should never be touched, AI-generated command or not.
- D is wrong: AI assistance can be a legitimate time-saver when used carefully; the module's AI angle sections describe exactly this kind of safe use.

Q27. A tester pastes a short packet capture line into an LLM and asks it to identify the TCP flags and the likely OS of the responding host, based on TTL and window size. The LLM answers confidently with a specific OS and version. What is the MOST appropriate next step?

- A. Immediately report the LLM's answer as a confirmed fact in the final report
- B. Cross-check the LLM's TTL/window reasoning against the reference table in this module, and confirm with an independent method such as -sV or banner grabbing before relying on it
- C. Ignore the LLM's answer entirely, since AI tools are never useful for this kind of task
- D. Re-run the same prompt ten times and average the answers

Answer: B. The module's AI angle notes throughout stress using AI output as a lead to verify, not as a substitute for the reference data and independent confirmation methods already covered (TTL/window table in 3.8, service and OS detection in 3.7 and 3.13).

- A is wrong: a single unverified claim, from a human or an AI, should not become a reported fact without confirmation.
- C is wrong: the module explicitly shows useful AI uses when the output is checked, not blanket dismissal.
- D is wrong: averaging repeated guesses from the same model does not substitute for checking against real reference data or an independent scan.

Q28. Which of the following BEST completes this sentence: "Service version detection (-sV) matters to an attacker mainly because..."

- A. it makes the scan run faster than a plain port scan
- B. it reveals the exact product and version behind an open port, which can be matched to known exploits

- C. it automatically exploits any vulnerable service it finds
- D. it replaces the need for host discovery

Answer: B. The module states plainly that obtaining an accurate service version number allows attackers to determine the vulnerability of a target system to particular exploits.

- A is wrong: version detection is slower than a plain port scan, not faster (46.55 seconds versus 1.65 seconds in the module's own example).
- C is wrong: -sV only identifies versions; it does not exploit anything by itself.
- D is wrong: version detection happens after host discovery and port scanning, not instead of them.

Q29. A firewall administrator wants to confirm that her organization's perimeter correctly drops packets whose source address falsely claims to be from inside the organization's own network. Which specific control is she testing?

- A. Egress filtering
- B. Ingress filtering
- C. Random initial sequence numbers
- D. Multiple firewalls

Answer: B. Ingress filtering specifically checks incoming traffic and drops packets that falsely claim an internal source address, which is exactly the scenario described.

- A is wrong: egress filtering checks the opposite direction, outgoing traffic leaving the network.
- C is wrong: random ISNs address blind TCP sequence-prediction spoofing, not address-origin checks at the perimeter.
- D is wrong: multiple firewalls add general defense in depth, but this question describes one specific filtering control, not a layering strategy.

Q30. Which statement correctly summarizes the relationship between TCP window scan (-sW) and TCP ACK scan (-sA)?

- A. They are unrelated scan types that use completely different probe packets
- B. The window scan sends the same type of ACK probe as the ACK scan, but additionally reads the TCP window field of the RST reply to sometimes distinguish open from closed ports
- C. The window scan is only usable on UDP services
- D. The ACK scan is always more accurate than the window scan

Answer: B. Both scans send an ACK probe; the window scan adds the extra step of checking whether the RST reply's window field is positive (suggesting open) or zero (suggesting closed), a refinement the plain ACK scan does not attempt.

- A is wrong: they use the exact same underlying ACK probe packet.

- C is wrong: both are TCP scan types, not UDP.
- D is wrong: accuracy depends on the target OS's implementation quirks; neither scan is universally more accurate than the other.

10. Flashcards

Q: What is network scanning? | A: Procedures used for identifying hosts, ports, and services in a network; phase 2 of the five hacking phases.

Q: What are the six TCP flags? | A: URG, ACK, PSH, RST, SYN, FIN.

Q: What does the SYN flag do? | A: Initiates a connection between hosts.

Q: What does the FIN flag do? | A: Signals there will be no further transmissions, ending a connection politely.

Q: What does RST do, compared to FIN? | A: RST abruptly resets/kills a connection at once; FIN ends it politely with further ACKs.

Q: How many packets make up the three-way handshake? | A: Three: SYN, SYN+ACK, ACK.

Q: How many packets make up TCP session termination? | A: Four: FIN, ACK, FIN, ACK.

Q: Which Nmap flag does an ICMP echo ping? | A: -PE.

Q: Which Nmap flag does an ARP ping? | A: -PR (most reliable on a local LAN).

Q: Which Nmap flag does a UDP ping? | A: -PU.

Q: Which Nmap flag does an IP protocol ping? | A: -PO.

Q: What does -sn do? | A: Ping scan only; skips port scanning.

Q: What does -Pn do? | A: Skips host discovery; treats every target as up.

Q: Which scan type completes the full TCP handshake? | A: TCP connect scan, -sT.

Q: Which scan type is "half-open" and needs root? | A: TCP SYN scan, -sS.

Q: Which three flags make up an Xmas scan probe? | A: FIN, PSH, and URG.

Q: What flags does a NULL scan send? | A: None at all.

Q: What flags does a Maimon scan send? | A: FIN and ACK.

Q: For FIN, NULL, Xmas, and Maimon scans, what does silence mean? | A: The port is open or filtered.

Q: Which major OS breaks FIN/NULL/Xmas/Maimon scans by answering RST to everything? | A: Windows (also many Cisco devices, BSDI, IBM OS/400).

Q: What can an ACK scan (-sA) never determine? | A: Whether a port is open; it only shows unfiltered or filtered.

Q: What does the window scan (-sW) add beyond a plain ACK scan? | A: It reads the TCP window field of the RST reply to sometimes tell open from closed.

Q: What is an idle (zombie) scan used for? | A: Hiding the real scanner's IP by using a third-party zombie host's IP ID counter to read scan results.

Q: In an idle scan, what IP ID rise means the port is open? | A: A rise of 2.

Q: What reply indicates a closed UDP port? | A: An ICMP port unreachable message (type 3, code 3).

Q: What do SCTP INIT and COOKIE ECHO scans correspond to? | A: -sY (INIT, like SYN scan) and -sZ (COOKIE ECHO, like FIN/NULL scans).

Q: What does -sV detect? | A: The service name and version running on an open port.

Q: What does -O do? | A: Active OS fingerprinting by sending a series of crafted probes.

Q: What TTL and window size suggest Windows? | A: TTL 128.

Q: What TTL and window size suggest Linux? | A: TTL 64, window size 5840.

Q: What are the two types of banner grabbing? | A: Active (crafted packets compared to a database) and passive (error messages, sniffed traffic, URL extensions).

Q: Which URL extension suggests IIS and Windows? | A: .aspx.

Q: What is packet fragmentation used for? | A: Splitting a probe's TCP header across several small IP fragments so simple filters cannot see the full header.

Q: What is source routing? | A: The sender specifies part or all of a packet's route through the network, bypassing firewalled routers.

Q: What is source port manipulation? | A: Setting the scanner's source port to a trusted well-known port (such as 80 or 53) to pass firewall rules that trust it.

Q: What is the difference between IP decoy and IP spoofing? | A: Decoy adds fake source IPs alongside the real scanner (result still readable); spoofing replaces the real source entirely (reply usually not seen).

Q: Why can't you complete a handshake with a spoofed source IP? | A: The target's SYN/ACK goes to the spoofed address, not back to the real attacker.

Q: Why did --spoof-mac fail with -sT in the module's example? | A: -sT uses the OS's own socket connection, not Nmap's raw packet engine that --spoof-mac needs.

Q: What does --randomize-hosts do? | A: Shuffles the order hosts in a range are scanned, to avoid an obvious sequential pattern.

Q: What does --badsum do, and what does a reply reveal? | A: Sends invalid checksums; any reply reveals a firewall/IDS answering on the target's behalf, since a real stack drops bad-checksum packets.

Q: Name two of the ten techniques for scanning beyond IDS and firewall. | A: Any two of: packet fragmentation, source routing, source port manipulation, IP address decoy, IP address spoofing, MAC address spoofing, creating custom packets, randomizing host order, sending bad checksums, proxy servers, anonymizers.

Q: What is ingress filtering? | A: Dropping incoming packets that falsely claim an internal source address.

Q: What is egress filtering? | A: Dropping outgoing packets whose source address is not actually part of your own network.

Q: Name two IP spoofing countermeasures besides filtering. | A: Encrypt traffic (IPsec/TLS/SSH/HTTPS) and use random initial sequence numbers; also do not rely on IP-based authentication.

11. Glossary

- **ACK (Acknowledgement):** TCP flag that confirms receipt of data.
- **Active banner grabbing:** sending crafted packets and comparing responses to a fingerprint database to identify an OS.
- **ACK scan:** a TCP scan that maps firewall rules; can only report unfiltered or filtered, never open.
- **Anonymizer:** a service or network (such as Tor) that hides a user's identity, often through multiple relay hops.
- **ARP (Address Resolution Protocol):** a local-network protocol that resolves an IP address to a MAC address.
- **Bad checksum:** a deliberately invalid TCP/UDP/SCTP checksum sent to test whether a firewall or IDS answers without validating it.
- **Banner:** text a service sends on connection, often naming its product and version.
- **Banner grabbing:** determining a target's OS or service by reading its responses, actively or passively.
- **Connect scan:** a TCP scan that completes the full three-way handshake; -sT in Nmap.
- **Decoy:** a fake source IP address added to a scan alongside the real one.
- **Egress filtering:** dropping outgoing packets whose source address is not genuinely part of your own network.
- **FIN (Finish):** TCP flag signalling no further transmissions.
- **FIN scan:** a scan that sends only the FIN flag; silence means open or filtered.
- **Fragmentation (packet):** splitting a probe packet into smaller IP fragments to evade simple filters.
- **Hping3:** a tool for crafting and sending custom TCP, UDP, ICMP, or raw IP packets.
- **IDS (Intrusion Detection System):** a device or program that watches traffic and alerts on attack patterns.
- **Idle scan:** a scan that spoofs probes through a "zombie" host and reads its IP ID counter to learn results blindly.
- **Ingress filtering:** dropping incoming packets that falsely claim an internal source address.
- **IP address decoy:** generating or specifying fake source IPs so the real scanner blends into a crowd.
- **IP address spoofing:** changing the source IP address so traffic appears to come from someone else.
- **IP option:** an optional field in the IP header, including source routing instructions.
- **IP protocol ping:** a host discovery probe that sets a chosen protocol number in the IP header.
- **ISN (Initial Sequence Number):** the starting sequence number a TCP stack picks for a new connection.

- **MAC address spoofing:** faking the hardware (MAC) address used in outgoing frames.
- **Maimon scan:** a scan sending FIN+ACK; relies on some BSD-derived systems failing to reply when a port is open.
- **NetBIOS:** a legacy Windows networking service, seen on port 139.
- **NULL scan:** a scan sending no TCP flags at all; silence means open or filtered.
- **OUI (Organizationally Unique Identifier):** the first three bytes of a MAC address, identifying the manufacturer.
- **Packet crafting tool:** software for building a network packet field by field, such as Colasoft Packet Builder.
- **Passive banner grabbing:** reading error messages, sniffed traffic, or URL extensions to learn a target's OS, without sending special probes.
- **Ping sweep:** sending the same discovery probe to every address in a range.
- **Proxy server:** a middle system that relays a connection, hiding the original source.
- **RST (Reset):** TCP flag that abruptly ends a connection.
- **SCTP (Stream Control Transmission Protocol):** a transport protocol used for telephony signalling, with its own INIT/COOKIE handshake.
- **Service version detection:** probing open ports to learn the exact software and version running; Nmap's -sV.
- **Source port manipulation:** setting a scanner's source port to a trusted well-known number to bypass a firewall rule.
- **Source routing:** specifying some or all of a packet's route through the network, bypassing certain routers.
- **SSDP (Simple Service Discovery Protocol):** the UPnP discovery protocol, running on UDP port 1900.
- **SYN (Synchronize):** TCP flag that initiates a connection.
- **SYN scan:** a half-open TCP scan that never completes the handshake; Nmap's -sS.
- **TCP (Transmission Control Protocol):** the reliable, connection-based transport protocol of the internet.
- **TTL (Time To Live):** a counter in the IP header that decreases at every router hop; different OS defaults help fingerprint a host.
- **UDP (User Datagram Protocol):** a connectionless transport protocol with no handshake.
- **UDP scan:** a scan that sends UDP datagrams; ICMP port unreachable means closed, no reply means open or filtered.
- **URG (Urgent):** TCP flag marking data for immediate processing.
- **Window scan:** an ACK-style scan that reads the TCP window field of RST replies to sometimes distinguish open from closed ports.
- **Xmas scan:** a scan sending FIN, PSH, and URG together; silence means open or filtered.
- **Zenmap:** the official graphical front end for Nmap.
- **Zombie (idle scan):** an idle host with a predictable IP ID counter, used to relay an idle scan's results.

12. Coverage Report

12.1 Page to Section Map

PDF page	Slides (topics)	Guide sections
1	2 objectives, 4 overview, 5 TCP flags	3.1, 3.2, 3.3
2	6 TCP/IP communication, 8 host discovery	3.4, 3.5
3	11 port scanning techniques, 15 service version discovery	3.6, 3.7
4	19 identify target OS, 26 packet fragmentation	3.8, 3.9
5	28 source port manipulation, 30 IP address spoofing	3.10, 3.11
6	32 creating custom packets, 18 OS discovery/banner grabbing	3.12, 3.13
7	25 scanning beyond IDS and firewall, 27 source routing	3.14, 3.15
8	29 IP address decoy, (unlabeled) MAC address spoofing	3.16, 3.17
9	33 randomize hosts/bad checksums, 37 port scan countermeasures	3.18, 3.19
10	42 IP spoofing countermeasures	3.20

Tools, commands, ports, and terms from all pages also appear in sections 4 to 11.

12.2 [UNCLEAR] Items

1. Page 1: the exact small print under "Overview of Network Scanning" diagram icons (attacker/network labels) and the faint strikethrough caption "we're gonna start with just what scanning is" are video captions, not slide content; included for completeness only.
2. Page 2: the exact command lines and full terminal text in the "TCP SYN Ping Scan" and "ICMP ECHO Ping Sweep" screenshots.
3. Page 3: the grouping lines connecting Maimon, TTL-based, and Window scans to their parent categories in the port scanning technique tree; also the exact date/time text in the service version discovery screenshot.
4. Page 4: the two Wireshark capture screenshots' exact highlighted byte values for "Possible OS is Windows" and "Possible OS is Linux"; the exact warning text and command in the packet fragmentation Zenmap screenshot.

5. Page 5: the diagram labels in the IP Address Spoofing slide, where both the spoofed-address box and a "real address" box are marked 7.7.7.7, which appears internally inconsistent as captured.
6. Page 6: the exact field values entered in the Colasoft Packet Builder Decode Editor, Hex Editor, and Packet List panes.
7. Page 8: the slide title and bullet text for the MAC Address Spoofing screenshot (no title or body text is visible in the capture; the topic was reconstructed from the --spoof-mac command and the "MAC Address Spoofing" icon named on slide 25).
8. Page 9: the exact timestamp in the --badsum screenshot.

12.3 Items to Double Check

- The TTL-based ACK scan (nmap -sA --ttl 100) and the window-based distinction are EC-Council courseware explanations; Nmap's own official manual documents the ACK scan and the window scan but does not describe a dedicated "TTL-based scan" as a distinct built-in mode, so treat the TTL rule of thumb as exam theory and verify its practical reliability before depending on it in a real engagement.
- Exact Nmap default ports for -PS, -PA, and -PU host discovery probes (commonly cited as 80 for -PS/-PA and 40125 for -PU), and the exact default decoy group behavior with -D RND:N, should be checked against your installed Nmap version's documentation.
- hping3 flag names (-a/--spoof, -S, -1/--icmp, -2/--udp, -8/--scan, -t/--ttl) were confirmed against the hping3 manual page; confirm behavior on your specific build, since very old or patched versions can differ slightly.
- --spoof-mac's exact interaction with each scan type (confirmed failing with -sT in the module's own screenshot) should be checked against -sU and -sY/-sZ behavior if you rely on it in those modes.
- The "SSDP Scanning" and "Proxy Servers"/"Anonymizers" subsections are ADDED CONTEXT, reconstructed from an icon label with no dedicated detail slide in the screenshots; verify current tool names and commands before using them in a real assessment.

12.4 Confirmation

Every line of the working inventory (pages 1 to 10, 19 numbered slides) is mapped to a section in the table above. Items that could not be read with certainty are listed in 12.2 with a best reading, and knowledge added from outside the screenshots is labeled ADDED CONTEXT throughout sections 3 to 8.